

MGT 4050 Final Project Report

Team Members

1. Serena Yin
2. Winnie Zheng
3. Jane (Yijun) Sun
4. Sophie Hollowell

Overall, our approach to this project was to divide and conquer. Each team member was responsible for specific parts of each phase of the project: from exploratory analysis to creating the presentation, resulting in a fair distribution of work. When we first began planning this project, we wanted to break down the predictor variables relationship with the response variable “*cnt*”. Therefore, for exploratory analysis, we distributed the predictor variables among the group members. Sophie was responsible for exploring “*weather_code*” and “*season*” as well as creating a correlation matrix for all predictor variables on the response variable. Serena looked at the predictor variables “*timestamp*”, “*is_holiday*”, and “*is_weekend*”; Jane observed the predictor variables “*t1*” and “*t2*”; and Winne looked at “*hum*” and “*windspeed*”. In addition to this initial exploratory analysis, our group members also did additional exploratory analysis looking at different combinations of variables and their effect on the response “*cnt*”. After taking individual time to complete this work, our group compiled insights to answer the questions in the Exploratory Analysis section of the project, and then, we discussed which potential predictive models we wanted to consider from class discussion. Because we wanted to explore as many of the options that we talked about in class, we decided on Decision Trees, Clustering & Association, Logistic Regression, and Linear Regression. Winnie built our Decision Trees Model; Serena created our Clustering & Association Model; Jane worked on our Logistic Regression Model; and Sophie created the Linear Regression Model. After creating this models and writing out our insights/analysis in a shared Google Doc, we came to the conclusion that Logistic Regression and Clustering were not appropriate while also deciding to explore Polynomial Regression. Jane and Sophie worked together to create the Polynomial Model. In terms of creating the Class Presentation, Sophie covered Background and Exploratory Analysis; Serena went over our Model Construction; Winnie discussed Evaluation Metrics; and Jane summarized our Findings and Business Recommendations. Throughout the duration of the project, every member has taken a significant part in completing this project and has had a hand in working on the technical exploration of data and creation of models as well as keeping each other member up to date about their part. All members have attended team meetings as we have set them, and each person has been responsible for compiling written reports of their insights.

Dataset Description

Predictor Variables (Y variables)

- “*timestamp*” - provides date of each record (i.e. 2015-01-04); broken down into variables “*month*”, “*day*”, and “*hour*”
- “*t1*” - real temperature in Celsius
- “*t2*” - “feels like” temperature in Celsius
- “*hum*” - humidity in percentage
- “*windspeed*” - wind speed in km/h
- “*weather_code*” - category of the weather (1 - clear, 2 - scattered clouds, 3 - broken clouds, 4 - cloudy, 7 - light rain, 10 - rain with thunderstorm, 26 - snowfall)
- “*is_holiday*” - binary variable where 1 is holiday and 0 is non-holiday
- “*is_weekend*” - binary variable where 1 is weekend and 0 is non-weekend
- “*season*” - category of the season (0 - spring, 1 - summer, 2 - fall, 3 - winter)

Response Variable (X variable)

- “*cnt*” - The count of new bike shares

Within this dataset, we are looking at the above predictor variables to see if there are any relationships or data insights that significantly improve the count of new bike shares. In understanding the predictor variables effect on the response variable “*cnt*”, we will have a better understanding of how to increase the number of new bike shares for a bike sharing company, ultimately improving that company’s core business activity.

Our Goals

1. Understand the relationship of the predictor variables on the response variable.
2. Create an appropriate prediction model to estimate the number of new bike shares.

In this project, we wanted to understand the relationship of the predictor variables on the response variable through our exploratory analysis. Specifically, we were looking to understand the effect of time, weather, and temperature on the number of new bike shares and hoping to answer the following questions:

- Which feature is the most associated with the number of bike shares?
- What bike-sharing patterns do you observe at different times of day, days of week, month, or holiday?

- What is the relation between weather situations and the number of bike shares? And what are your explanations?

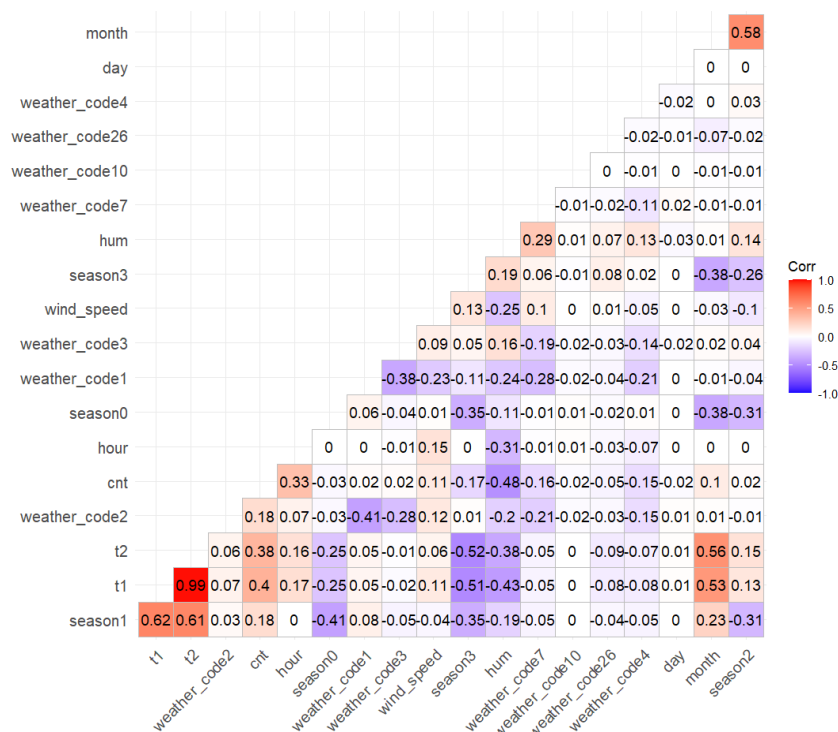
By understanding these relationships, we wanted to create a prediction model to estimate the number of new bike shares and understand how a bike sharing company could best maximize new bike sharing. Ultimately, we were trying to answer: “How can a bike share company increase profit through increasing the number of bike shares and what factors should a bike share company consider when developing its business strategy?”

Exploratory Analysis

1. Which feature is the most associated with the number of bike shares?

Based on the correlation matrix below, we see that “hum” has the correlation at -0.48.

“t1” has the second highest correlation at 0.4, and “t2” has the third highest correlation at “0.38”.

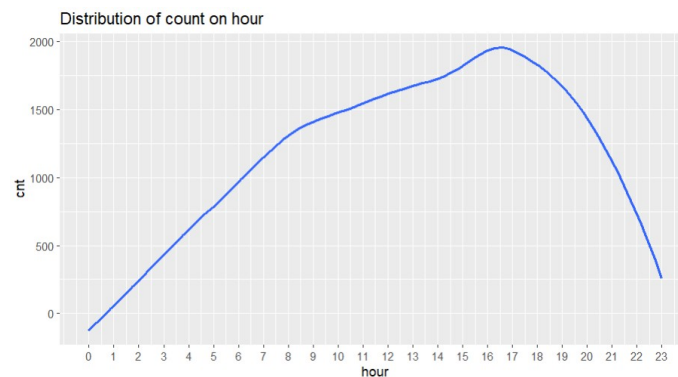


2. What bike-sharing patterns do you observe at different times of day, days of week, month, or holiday?

The number of bike shares tend to increase towards the afternoons and decrease during nighttime. On average, the peak of bike shares usually occurs between 4-5PM (16:00-17:00). A few causes for this peak could be the end of the workday and school dismissal where people are

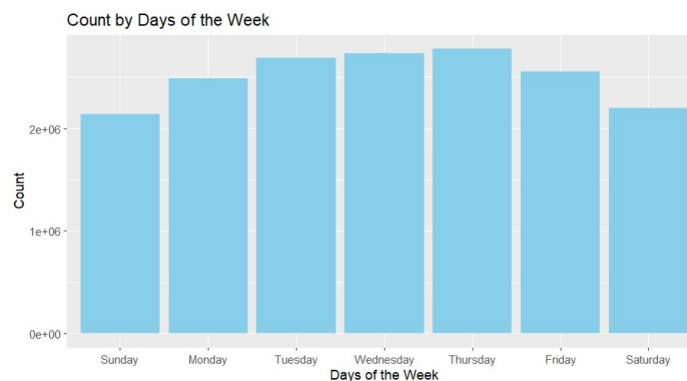
using bike sharing as a mode of transportation to get home. People could also be going to evening social engagements or relying more on bike sharing if other transit options are decreasing in availability or frequency. Furthermore, bike sharing could be more appealing due to nicer weather during these times.

```
ggplot(data, aes(x = hour, y = cnt)) +
  geom_smooth(method = "loess", se = FALSE) +
  labs(x = "hour", y = "cnt", title = "Distribution of count on hour") +
  scale_x_continuous(breaks = seq(0, 23, by = 1), labels = seq(0, 23, by = 1))
```



The number of bike shares is usually the highest during the middle of the week, which is Tuesday, Wednesday, and Thursday. Out of those days of the week, Thursday tends to have the most bike shares. A few potential reasons for people using bike sharing the most on these days could be that many people tend to run midweek errands in preparation for the weekend and public transportation could be the most crowded during these days.

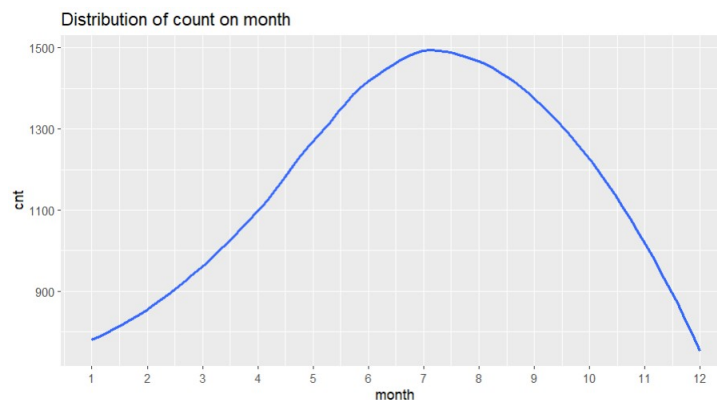
```
ggplot(data, aes(x = day_of_week, y = cnt)) +
  geom_bar(stat = "identity", fill = "skyblue") +
  labs(x = "Days of the Week", y = "Count", title = "Count by Days of the Week")
```



The number of bike shares tends to increase from the months of January to July, until it reaches the peak in July. Afterwards, the count of bike shares steadily decreases until December.

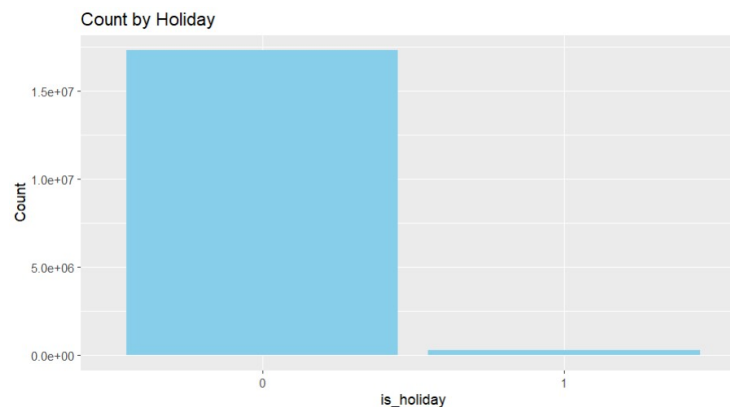
This increase towards the summer marks likely derives from nicer weather, summer activities, tourism, or a health & fitness focus during summer months. Furthermore, the decrease moving towards December is likely from less optimal bike riding weather: cold weather is not as appealing to ride a bike in and any ice or rain results in dangerous bike riding conditions.

```
ggplot(data, aes(x = month, y = cnt)) +  
  geom_smooth(method = "loess", se = FALSE) +  
  labs(x = "month", y = "cnt", title = "Distribution of count on month") +  
  scale_x_continuous(breaks = seq(1, 12, by = 1), labels = seq(1, 12, by = 1))
```



People tend to not share bikes on holidays, which makes sense because holidays prompt need for longer distance travel as well as reduced work & school commute that many people might use bike sharing for.

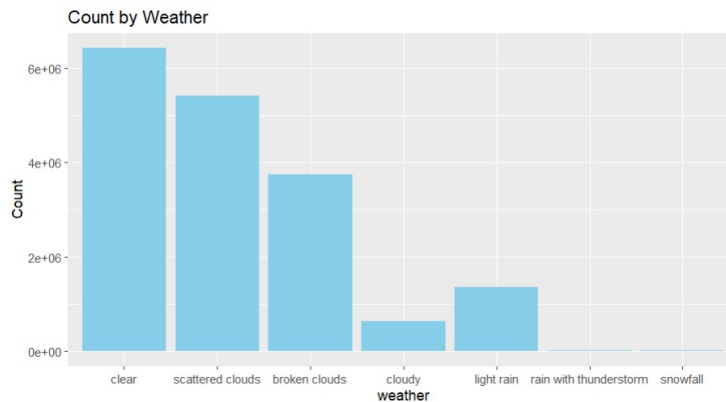
```
ggplot(data, aes(x = is_holiday, y = cnt)) +  
  geom_bar(stat = "identity", fill = "skyblue") +  
  labs(x = "is_holiday", y = "Count", title = "Count by Holiday")
```



3. What is the relation between weather situations and the number of bike shares? And what are your explanations?

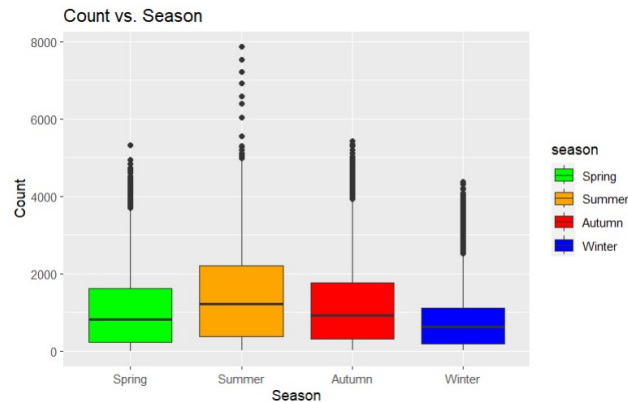
The number of bike shares is higher for weather situations with the potential of sunshine, such as clear, scattered clouds, and broken clouds. Typically on those days, there are favorable bike riding conditions of sunlight, visibility, and warmer temperatures. In contrast, when there is rain with thunderstorms or snowfall, there are rarely any bike shares. The rain and snow makes bike riding conditions dangerous, uncomfortable, and unpleasant, as the bike riders will be cold and wet.

```
ggplot(data, aes(x = weather_code, y = cnt)) +  
  geom_bar(stat = "identity", fill = "skyblue") +  
  labs(x = "weather", y = "Count", title = "Count by Weather")
```



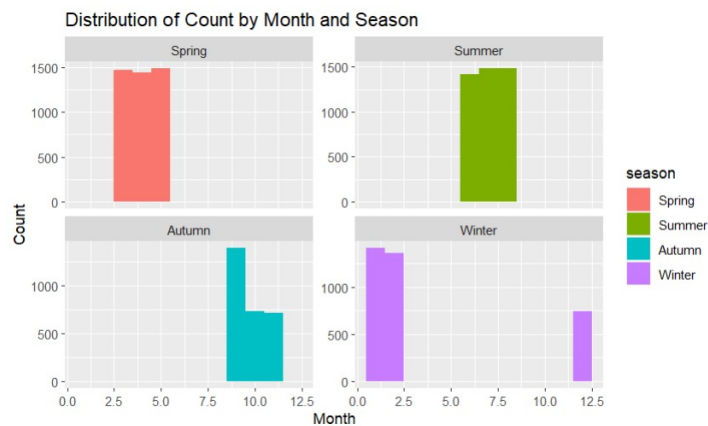
Furthermore, we wanted to explore the variable “season” to see if it had any impact on the weather situation and number of bike shares. The graph below shows that the number of bike shares are the highest in the Summer and lowest in the Winter. In descending order of bike share counts: Summer, Fall, Spring, and Winter.

```
ggplot(data, aes(x = season, y = cnt, fill = season)) +  
  geom_boxplot() +  
  labs(x = "Season", y = "Count", title = "Count vs. Season") +  
  scale_fill_manual(values = c("Spring" = "green", "Summer" = "orange", "Autumn" =  
    "red", "Winter" = "blue"))
```



Furthermore, we can also see that certain months are correlated with certain seasons of the year, which makes sense because during specific months of the year, we expect a specific season.

```
ggplot(data, aes(x = month, fill = season)) +  
  geom_histogram(binwidth = 1, position = "dodge") +  
  labs(x = "Month", y = "Count", title = "Distribution of Count by Month and Season") +  
  facet_wrap(~ season, scales = "free_y")
```



Summer has the highest temperatures. As temperature (t1) increases, the marginal effect of Summer on the count of bike shares increases. In other seasons, the marginal effect of temperature on the count of bike shares is lowered

As the weather code increases (i.e., the weather conditions become worse from clear to snowfall), the marginal effect of temperature (t1) on the count of bike shares is lowered

The correlation between t1 (real temperature) and t2 ("feels like" temperature) is approximately 0.988, there is a strong correlation between these two variables. Furthermore, the high VIF values of 44.94664 for both t1 and t2 in the regression model $\text{lm}(\text{cnt} \sim \text{t1} + \text{t2})$ suggest that multicollinearity is present.

```

> lm.temp <- lm(cnt~t1+t2, data = data)
> vif(lm.temp)
      t1      t2
44.95422 44.95422

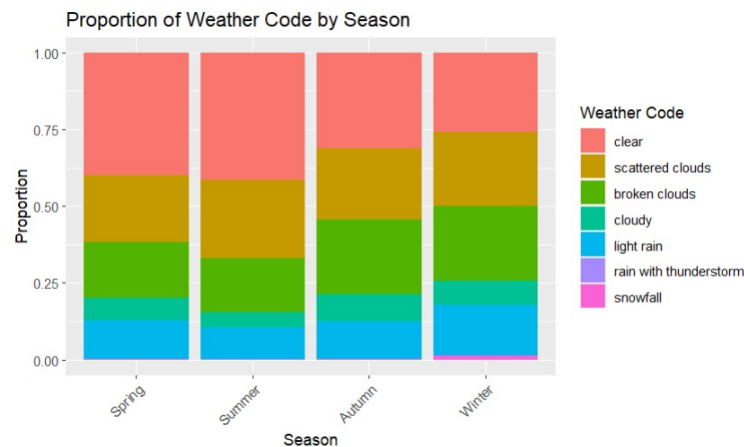
```

The occurrences of the different weather situations are slightly correlated with the different seasons based on the map below; therefore, it is also correlated with the months of the year. For instance, the occurrence of snowfall only happens in the Winter, or there is more clear weather during the Summer. However, we do not see significant changes in the weather situation based on season; generally, it looks like the proportions stay relatively consistent across season and month. For example, across seasons, we do not see that one weather situation becomes more prominent, and generally, the proportion of weather situations is similar for each season, with the exception of snowfall which only occurs during winter.

```

ggplot(data, aes(x = season, fill = weather_code)) +
  geom_bar(position = "fill") +
  labs(x = "Season", y = "Proportion", title = "Proportion of Weather Code by Season") +
  scale_fill_discrete(name = "Weather Code") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

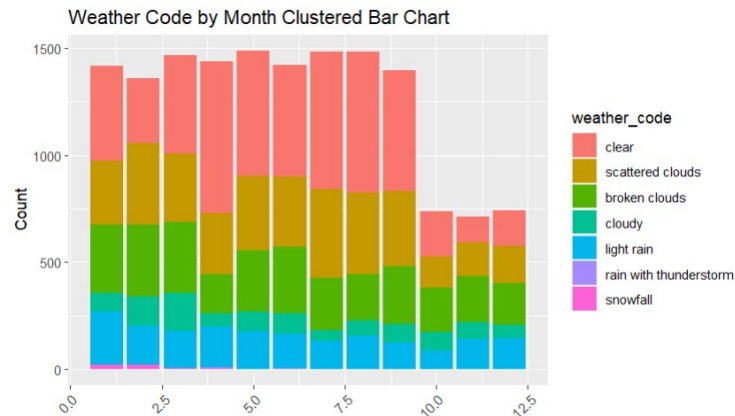
```



```

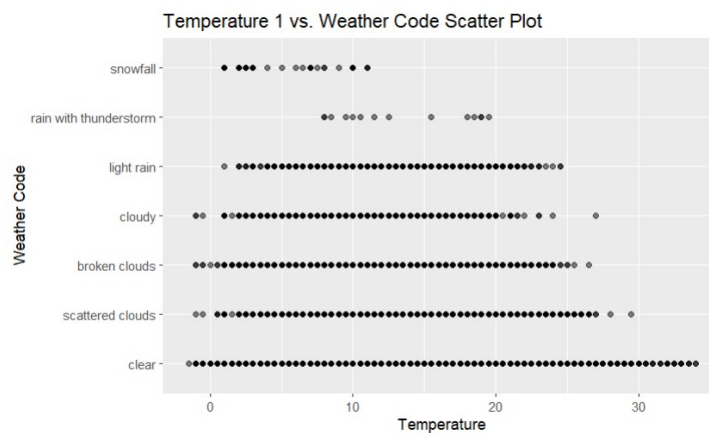
ggplot(data, aes(x = month, fill = weather_code)) +
  geom_bar() +
  labs(x = "Month", y = "Count", title = "Weather Code by Month Clustered Bar Chart")
+
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

```

As t_1 increases, the marginal effect of weather_code on the count of bike share is lowered. Temperature and weather_code seems to have no correlation, as most of the weather situations occur regardless of the temperature. However, snowfall only occurs at below 15°C.

```
ggplot(data, aes(x = t1, y = weather_code)) +
  geom_point(alpha = 0.5) +
  labs(x = "Temperature", y = "Weather Code", title = "Temperature 1 vs. Weather Code Scatter Plot")
```



Ultimately, it seems like temperature, season, and month do not have a material impact on weather situations. Therefore, better bike riding weather conditions alone majorly account for increases in number of bike shares while less favorable weather conditions, such as rain and snow, prompt significant decrease in number of bike shares.

Predictive Model Construction

Logistic Regression Model

Before fitting the model, we converted all categorical variables – “isholiday”, “isweekend”, “weathercode”, and “season” – to factor variables; converted timestamp to POSIXct format to month, day, and hour to better predict ‘cnt’(same for testing_data):

```
my_data$weather_code <- as.factor(my_data$weather_code)
my_data$season <- as.factor(my_data$season)
my_data$isholiday <- as.factor(my_data$is_holiday)
my_data$isweekend <- as.factor(my_data$is_weekend)
my_data$timestamp <- as.POSIXct(my_data$timestamp, format = "%Y-%m-%d %H:%M:%S")
my_data$humidity <- as.numeric(my_data$hum)

my_data$month <- factor(month(my_data$timestamp))
my_data$day <- factor(day(my_data$timestamp))
my_data$hour <- factor(hour(my_data$timestamp))
str(my_data)
```

Using the logistic regression model to predict the number of new bike shares (the “cnt” variable) is not possible because this particular regression model is typically used for classification problems, particularly binary classification, rather than predicting quantitative values. However, we still wanted to use the logistic regression model to explore the data range or category of ‘cnt’. We used two ways to categorize ‘cnt’, after scaling the data:

- 1) binary: low demand (lower than median) and high demand (higher than median)
- 2) advanced categories of four levels: "Very Low", "Low", "High", and "Very High"

```
# Create a binary outcome variable for high demand days
median_cnt <- median(my_data$cnt)
my_data$high_demand <- as.factor(ifelse(my_data$cnt > median_cnt, 1, 0))
testing_data$high_demand <- as.factor(ifelse(testing_data$cnt > median_cnt, 1, 0))

# logistic regression model
logit_model <- glm(high_demand ~ t1 + t2 + hum + wind_speed + weathercode + isholiday +
  isweekend + season + month + day + hour,
  family = binomial(link = "logit"), data = my_data)
logit_model
```

summary(logit_model)

Call:

```
glm(formula = high_demand ~ t1 + t2 + hum + wind_speed + weathercode +  
    isholiday + isweekend + season + month + day + hour, family = binomial(link = "logit"),  
    data = my_data)
```

Coefficients: (3 not defined because of singularities)

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-1.959036	0.550350	-3.560	0.000371 ***
t1	0.281022	0.049201	5.712	1.12e-08 ***
t2	-0.069050	0.037397	-1.846	0.064831 .
hum	-0.059713	0.003711	-16.092	< 2e-16 ***
wind_speed	-0.052492	0.005056	-10.382	< 2e-16 ***
weathercode2	0.070303	0.098690	0.712	0.476241
weathercode3	-0.025402	0.102112	-0.249	0.803542
weathercode4	-0.124063	0.146889	-0.845	0.398332
weathercode7	-1.689463	0.126700	-13.334	< 2e-16 ***
weathercode10	-4.334148	0.855698	-5.065	4.08e-07 ***
weathercode26	0.281596	0.678384	0.415	0.678070
isholiday1	-1.966095	0.209833	-9.370	< 2e-16 ***
isweekend1	-1.621196	0.073640	-22.015	< 2e-16 ***
season1	-0.254732	0.185756	-1.371	0.170274
season2	0.008688	0.179208	0.048	0.961336
season3	-0.377991	0.171304	-2.207	0.027345 *
month2	0.210825	0.126451	1.667	0.095465 .
month3	-0.160445	0.165776	-0.968	0.333124
month4	0.029600	0.161261	0.184	0.854364
month5	NA	NA	NA	NA
month6	0.591018	0.181096	3.264	0.001100 **
month7	0.558902	0.180838	3.091	0.001997 **
month8	NA	NA	NA	NA
month9	-0.216018	0.190106	-1.136	0.255830
month10	0.219536	0.200919	1.093	0.274545
month11	NA	NA	NA	NA
month12	-0.266128	0.164696	-1.616	0.106121
day2	-0.304595	0.250630	-1.215	0.224245
day3	-0.105916	0.258622	-0.410	0.682142
day4	-0.077366	0.255843	-0.302	0.762350
day5	-0.226080	0.250317	-0.903	0.366434
day6	-0.086780	0.249690	-0.348	0.728178
day7	-0.256762	0.254389	-1.009	0.312818
day8	-0.161227	0.255815	-0.630	0.528532
day9	0.394133	0.261385	1.508	0.131590
day10	0.008293	0.252819	0.033	0.973833
day11	-0.161508	0.255913	-0.631	0.527971

day12	-0.218759	0.253621	-0.863	0.388389
day13	0.163872	0.257356	0.637	0.524288
day14	-0.102025	0.249272	-0.409	0.682325
day15	-0.027728	0.254403	-0.109	0.913209
day16	0.054971	0.253664	0.217	0.828434
day17	0.392383	0.260842	1.504	0.132506
day18	0.116703	0.251982	0.463	0.643265
day19	-0.048790	0.252403	-0.193	0.846724
day20	-0.006263	0.253282	-0.025	0.980272
day21	0.073006	0.249697	0.292	0.769999
day22	-0.053060	0.254954	-0.208	0.835140
day23	0.109698	0.264993	0.414	0.678900
day24	-0.569895	0.247416	-2.303	0.021257 *
day25	0.282718	0.256584	1.102	0.270525
day26	-0.397352	0.247459	-1.606	0.108333
day27	0.059943	0.260727	0.230	0.818165
day28	0.221961	0.252687	0.878	0.379725
day29	-0.318933	0.251178	-1.270	0.204174
day30	-0.209818	0.264427	-0.793	0.427497
day31	-0.662970	0.285019	-2.326	0.020015 *
hour1	-14.809085	371.703057	-0.040	0.968220
hour2	-14.644073	377.274960	-0.039	0.969038
hour3	-14.520442	378.832863	-0.038	0.969425
hour4	-14.439346	378.249355	-0.038	0.969549
hour5	-14.419271	379.352997	-0.038	0.969680
hour6	1.424049	0.441307	3.227	0.001251 **
hour7	6.945335	0.389539	17.830	< 2e-16 ***
hour8	6.744161	0.389346	17.322	< 2e-16 ***
hour9	7.500682	0.396511	18.917	< 2e-16 ***
hour10	6.501080	0.392331	16.570	< 2e-16 ***
hour11	5.947510	0.391543	15.190	< 2e-16 ***
hour12	7.792865	0.408193	19.091	< 2e-16 ***
hour13	7.647515	0.407614	18.762	< 2e-16 ***
hour14	6.921718	0.401420	17.243	< 2e-16 ***
hour15	7.492557	0.407344	18.394	< 2e-16 ***
hour16	9.061286	0.437183	20.727	< 2e-16 ***
hour17	9.047171	0.432333	20.926	< 2e-16 ***
hour18	8.493510	0.417534	20.342	< 2e-16 ***
hour19	7.658564	0.401882	19.057	< 2e-16 ***
hour20	5.974642	0.387469	15.420	< 2e-16 ***
hour21	4.025198	0.383200	10.504	< 2e-16 ***
hour22	2.881097	0.387589	7.433	1.06e-13 ***
hour23	1.376280	0.414171	3.323	0.000891 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 20980.2 on 15133 degrees of freedom

Residual deviance: 6466.4 on 15057 degrees of freedom

(2 observations deleted due to missingness)

AIC: 6620.4

Number of Fisher Scoring iterations: 18

```
> str(my_data$high_demand)
```

Factor w/ 2 levels "0","1": 1 1 1 1 1 1 1 1 1 1 ...

From the summary output, we can tell that t2, the days , months and seasons variables are not statistically significant and we can therefore assume that they do not have a strong association with predicting the demand.

Null vs. Residual Deviance: A large reduction from null deviance (model with only the intercept) to residual deviance (full model) indicates that the predictors collectively have a significant impact on the model.

After the logistic regression analysis, we used the model for prediction and evaluation. We wanted to use accuracy to evaluate the performance of this logistic regression model:

```
# Predictions and model evaluation  
# Predict probabilities  
predicted_probs <- predict(logit_model, newdata = testing_data, type = "response")  
predicted_probs  
  
# Convert probabilities to binary outcome based on threshold  
predicted_class <- ifelse(predicted_probs > 0.5, 1, 0)  
predicted_class
```

Convert to Binary Outcome: the predicted probabilities are then converted into a binary outcome based on a threshold of 0.5. If the predicted probability is greater than 0.5, it is classified as a high-demand day (1); otherwise, it is classified as a low-demand day (0).

```
# Confusion matrix  
confusion_matrix <- table(Predicted = predicted_class, Actual = testing_data$high_demand)  
confusion_matrix
```

```
> confusion_matrix  
      Actual  
Predicted  0    1  
      0 1118 173  
      1   95 892
```

```
# Calculate accuracy  
accuracy <- sum(diag(confusion_matrix)) / sum(confusion_matrix)
```

```
print(paste("Accuracy: ", accuracy))
```

```
> print(paste("Accuracy: ", accuracy))  
[1] "Accuracy:  0.88235294117647"
```

This shows the accuracy of the model is approximately 88.24%. This means that the model correctly predicts whether a day is a high-demand or low-demand day 88.24% of the time. This is quite a high rate, suggesting that the model is effective at distinguishing between high-demand and low-demand days for bike shares. This level of accuracy indicates that the model is reliable for making predictions on new data.

For our logistic regression model for advanced demand categories:

```
##-----Categorize 'cnt' to four levels: "Very Low", "Low", "High", and "Very High" -----###  
# Define quartiles as thresholds  
quartiles_cnt <- quantile(my_data$cnt, probs = c(0.25, 0.5, 0.75))  
  
# Categorize `cnt` into four levels based on quartiles  
my_data$demand_level <- cut(my_data$cnt, breaks = c(-Inf, quartiles_cnt, Inf), labels = c("Very  
Low", "Low", "High", "Very High"))  
testing_data$demand_level <- cut(testing_data$cnt, breaks = c(-Inf, quartiles_cnt, Inf), labels =  
c("Very Low", "Low", "High", "Very High"))  
  
library(nnet)  
# Train the multinomial logistic regression model  
multinom_model <- multinom(demand_level ~ t1 + t2 + hum + wind_speed + weathercode +  
isholiday + isweekend + season + month + day + hour, data = my_data)  
str(multinom_model)  
summary(multinom_model)
```

```

> summary(multinom_model)
Call:
multinom(formula = demand_level ~ t1 + t2 + hum + wind_speed +
          weathercode + isholiday + isweekend + season + month + day +
          hour, data = my_data)

Coefficients:
(Intercept)          t1          t2          hum  wind_speed weathercode2 weathercode3 weathercode4 weathercode7 weathercode10
Low      0.7116817 0.01923539 0.043027970 -0.02155057 -0.02480329 0.2142966 0.02014283 -0.2793028 -0.9725998 -2.983973
High     -0.6514542 0.26241247 -0.005371215 -0.07447726 -0.07549161 0.2737493 0.01687688 -0.4430632 -2.3575838 -6.394307
Very High -2.5400659 0.51112742 -0.134979497 -0.10051525 -0.09404502 0.1535278 -0.15926652 -0.3758550 -3.2949163 -11.018053
weathercode26 isholiday1 isweekend1 season1 season2 season3 month2 month3 month4 month5 month6
Low      -1.2381374 -0.6091245 0.187300 0.3819684 0.3100178 -0.1593188 0.10268612 -0.01913274 -0.08028662 0.27843366 0.06285509
High     0.4103321 -2.5694693 -1.395864 0.2136064 0.1233397 -0.5663544 0.38474473 -0.17559546 -0.23076641 -0.01568404 0.29097724
Very High -0.1502161 -2.4289219 -1.615227 0.1649165 -0.0416865 -1.0397405 0.01161629 -0.88135066 -0.56660600 -0.17559878 0.09091862
month7 month8 month9 month10 month11 month12 day2 day3 day4 day5 day6
Low      0.2289001 0.09021315 0.04494049 0.1551563 0.1099210 0.15120903 0.2188688 -0.02864471 0.2410655 0.3495659 0.2627054
High     0.2826088 -0.35997960 -0.28688804 0.2143087 0.1959190 0.04742042 -0.0806530 -0.16046002 0.2318081 0.1299861 0.1083646
Very High 0.4038634 -0.32986557 -0.30750936 0.4433410 -0.1775181 -1.01578587 0.1727735 0.22360750 0.2781516 0.2604384 0.5493260
day7 day8 day9 day10 day11 day12 day13 day14 day15 day16 day17 day18
Low      0.210597013 0.3080599 0.3119887 0.4926785 0.5519899 0.4959847 0.4354357 0.1715537 0.4392094 0.3607601 0.3979944 0.6795917
High     0.009262684 0.1772403 0.6682175 0.5425632 0.3371293 0.3302310 0.5713950 0.1434413 0.4392277 0.5000070 0.8059695 0.7477437
Very High 0.012535193 0.2247481 1.1132272 0.5839536 0.6899274 0.3470792 0.8017173 0.1139447 0.5978746 0.4438741 0.9800386 1.0909049
day19 day20 day21 day22 day23 day24 day25 day26 day27 day28 day29
Low      0.5693151 0.5121108 0.2906816 -0.12653510 0.2036576 0.2647177 0.2838031 0.34615538 0.3358825 0.04202224 0.24644164
High     0.5677325 0.5180600 0.4550537 -0.01181405 0.3828527 -0.2550842 0.5161527 -0.02124756 0.4696926 0.43795529 -0.01740817
Very High 0.4749899 0.5079190 0.2266699 -0.20282450 0.3298456 -0.2946848 0.7753841 0.03629405 0.2183514 -0.01150401 -0.09511258
day30 day31 hour1 hour2 hour3 hour4 hour5 hour6 hour7 hour8 hour9
Low      0.24018854 0.51956039 -1.193244 -1.6695387 -2.5860144 -14.8305735 -5.6525000 1.086785 -0.08051774 2.304317 4.192084
High     0.10913665 0.04618755 -2.725775 -2.6246292 -2.5858080 -2.6524626 -2.6023541 2.017843 5.69041957 4.734239 9.900929
Very High 0.01277414 -0.33622432 -0.616548 -0.3554735 -0.2641193 -0.3772435 -0.4343217 2.386085 10.26535232 11.853947 13.859341
hour10 hour11 hour12 hour13 hour14 hour15 hour16 hour17 hour18 hour19 hour20 hour21 hour22
Low      3.930045 3.783148 4.732428 4.11553 4.377193 3.809321 3.959737 4.178071 4.508112 4.361858 3.679445 3.437816 2.905150
High     9.582996 8.830355 11.492939 10.70290 10.285596 10.233669 11.717771 10.966206 10.677779 10.673437 8.771047 6.793990 5.206754
Very High 8.972000 9.621424 12.695274 12.20408 11.607762 12.027275 14.511981 15.858625 15.563781 13.511336 9.442675 5.245215 2.179447
hour23
Low      1.790108
High     2.738652
Very High 1.183513

Std. Errors:
(Intercept)          t1          t2          hum  wind_speed weathercode2 weathercode3 weathercode4 weathercode7 weathercode10
Low      0.3948697 0.05573852 0.04386865 0.004918931 0.006580802 0.1189614 0.1147621 0.1339598 0.1323428 0.74526838
High     0.5408219 0.07147647 0.05558296 0.005924161 0.007968411 0.1481150 0.1483072 0.1960942 0.1790223 1.00645239
Very High 0.4939652 0.07724131 0.06077752 0.006380340 0.008543370 0.1559694 0.1597023 0.2379259 0.2017128 0.01087914
weathercode26 isholiday1 isweekend1 season1 season2 season3 month2 month3 month4 month5 month6 month7
Low      0.54447614 0.2439610 0.08038196 0.1180756 0.0995954 0.1351520 0.1542511 0.1303738 0.1235207 0.1311865 0.1094544 0.1162378
High     0.8692571 0.3194485 0.10665354 0.1571796 0.1305652 0.1754556 0.1928361 0.1665130 0.1610013 0.1771043 0.1463506 0.1556839
Very High 1.1233367 0.3409072 0.11559554 0.1651988 0.1348700 0.1825596 0.2175960 0.1700918 0.1612302 0.1832016 0.1573304 0.1665060
month8 month9 month10 month11 month12 day2 day3 day4 day5 day6 day7 day8
Low      0.1167635 0.1289514 0.1407761 0.1391845 0.2093378 0.2392124 0.2370506 0.2372651 0.2350484 0.2317227 0.2337039 0.2362057
High     0.1578995 0.1658812 0.1809690 0.1757965 0.2569927 0.2760629 0.2761464 0.2736097 0.2673339 0.2668359 0.2692340 0.2711970
Very High 0.1679206 0.1761214 0.1915224 0.1923026 0.2931512 0.2924323 0.2939253 0.2933498 0.2905688 0.2876286 0.2938482 0.2929068
day9 day10 day11 day12 day13 day14 day15 day16 day17 day18 day19 day20
Low      0.2394236 0.2382648 0.2392805 0.2381880 0.2351628 0.2279677 0.2373832 0.2369759 0.2412195 0.2418684 0.2351168 0.2381584
High     0.2770221 0.2722225 0.2765306 0.2731536 0.2751826 0.2612141 0.2728695 0.2688621 0.2765354 0.2731649 0.2672347 0.2700008
Very High 0.2979310 0.2917935 0.2947191 0.2950841 0.3013022 0.2854117 0.2958974 0.2932271 0.2962770 0.2927062 0.2916941 0.2930770
day21 day22 day23 day24 day25 day26 day27 day28 day29 day30 day31 hour1
Low      0.2346195 0.2285673 0.2388066 0.2340735 0.2354026 0.2311525 0.2386670 0.2310407 0.2350963 0.2449305 0.2800325 0.1343526
High     0.2652518 0.2603290 0.2788749 0.2643827 0.2727713 0.2622996 0.2764215 0.2633673 0.2683241 0.2860805 0.3370147 1.0970132
Very High 0.2920740 0.2879049 0.3015962 0.2874968 0.2926804 0.2851303 0.2989996 0.2915289 0.2939643 0.3054898 0.3743477 0.8857421
hour2 hour3 hour4 hour5 hour6 hour7 hour8 hour9 hour10 hour11 hour12 hour13
Low      0.146586 0.1858602 1.028704e-05 0.7062920 0.1301650 0.1885503 0.2139158 0.4704404 0.3867565 0.3563588 0.6902029 0.5308171
High     1.082324 1.0541628 1.064371e+00 1.0815811 0.4497388 0.4052338 0.5494328 0.5986944 0.5314874 0.5105990 0.7811469 0.6448351
Very High 0.867258 0.8503568 8.761983e-01 0.9444917 0.6341020 0.2082797 0.2490961 0.4701896 0.4325980 0.3795938 0.6735289 0.5257448
hour14 hour15 hour16 hour17 hour18 hour19 hour20 hour21 hour22 hour23
Low      0.5113917 0.4852197 0.7121289 0.7134620 0.6924635 0.5257453 0.3238005 0.2360108 0.1863964 0.1411068
High     0.6309441 0.6082944 0.7961151 0.8046569 0.7903189 0.6420719 0.4862620 0.4327804 0.4129779 0.4205068
Very High 0.5108513 0.4848759 0.6860494 0.6898789 0.6743298 0.5203473 0.3543883 0.4541377 0.10658138 0.9162134

Residual Deviance: 17359.4
AIC: 17821.4

```

From the summary output, we can tell the reference category for ‘demand_verylow’ is presumably too low to show.

We proceeded to use the new model to predict:

```

# Predict the probabilities for levels of `demand_level`
predicted_probs_multinom <- predict(multinom_model, newdata = testing_data, type = "probs")

```

```

# Determine the class with the highest probability for each observation

```

```

predicted_classes_multinom <- apply(predicted_probs_multinom, 1, function(x)
names(which.max(x)))

# Convert to factor with the same levels as in the training data for consistency
predicted_classes_multinom <- factor(predicted_classes_multinom, levels =
levels(my_data$demand_level))

# Confusion Matrix
confusion_matrix_multinom <- table(Predicted = predicted_classes_multinom, Actual =
testing_data$demand_level)
print(confusion_matrix_multinom)
> print(confusion_matrix_multinom)
      Actual
Predicted Very Low Low High Very High
Very Low    507  70    9      14
Low         98 451  128     44
High         0  62  464     77
Very High    6  19   45    284

# Calculate accuracy
accuracy_multinom <- sum(diag(confusion_matrix_multinom)) /
sum(confusion_matrix_multinom)
print(paste("Accuracy: ", accuracy_multinom))
> print(paste("Accuracy: ", accuracy_multinom))
[1] "Accuracy:  0.748902546093064"

```

The model performs well with an accuracy of 74.89%, indicating a strong predictive capability.

Removing insignificant variables t2, season, month, day to test:

```

#remove t2, season, month, day
multinom_model <- multinom(demand_level ~ t1 + hum + wind_speed + weathercode +
isholiday + isweekend + hour, data = my_data)
str(multinom_model)
summary(multinom_model)

# Predict the probabilities for levels of `demand_level`

```



```

predicted_probs_multinom <- predict(multinom_model, newdata = testing_data, type = "probs")

# Determine the class with the highest probability for each observation
predicted_classes_multinom <- apply(predicted_probs_multinom, 1, function(x)
names(which.max(x)))

# Convert to factor with the same levels as in the training data for consistency
predicted_classes_multinom <- factor(predicted_classes_multinom, levels =
levels(my_data$demand_level))

# Confusion Matrix
confusion_matrix_multinom <- table(Predicted = predicted_classes_multinom, Actual =
testing_data$demand_level)
print(confusion_matrix_multinom)
> print(confusion_matrix_multinom)
      Actual
Predicted Very Low Low High Very High
Very Low   520  82   7      13
Low        90 434 116      35
High       0  58 475      83
Very High  1  28  48     288

```

Predicted \ Actual	Very Low	Low	High	Very High
Very Low	520	82	7	13
Low	90	434	116	35
High	0	58	475	83
Very High	1	28	48	288

```

# Calculate accuracy
accuracy_multinom <- sum(diag(confusion_matrix_multinom)) /
sum(confusion_matrix_multinom)
print(paste("Accuracy: ", accuracy_multinom))
> print(paste("Accuracy: ", accuracy_multinom))
[1] "Accuracy:  0.753731343283582"

```

The model performs well with an accuracy of 75.37%, indicating a good predictive capability. Removing the insignificant variables strengthens the prediction accuracy of the logistic regression model.

Clustering

K-Means Clustering

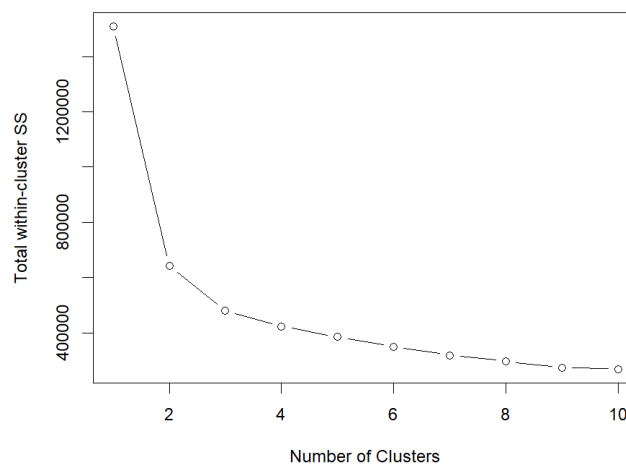
We utilized k-means clustering to identify trends between the different observations in the dataset. While k-means clustering could have been used to predict the count of bike shares by assigning the center of each cluster as the predicted value, doing so would result in predictions that are not particularly precise.

Instead, we visualized the clusters to better understand if there were any trends within the observations across predictor variables.

We first began by normalizing the test data to ensure that there was no single predictor variable that heavily influenced the clustering results because of its scale and unit of measure.

```
indicator_cols <- train_data[, c("is_holiday", "is_weekend", 'weather_code', 'season', 'month',  
'day')]  
numeric_cols <- train_data[, !(names(train_data) %in% c("is_holiday", "is_weekend",  
'weather_code', 'season', 'month', 'day'))]  
scaled_numeric <- scale(numeric_cols)  
train.data.sc <- cbind(scaled_numeric, indicator_cols)
```

To determine how many clusters to create, we ran k-means clustering for clusters sizes of 1 through 10. Taking the clustering results and the total within-cluster sum of squares, we created an elbow chart to identify the cluster size where the within-cluster sum of squares decreases less rapidly for a bigger cluster size.



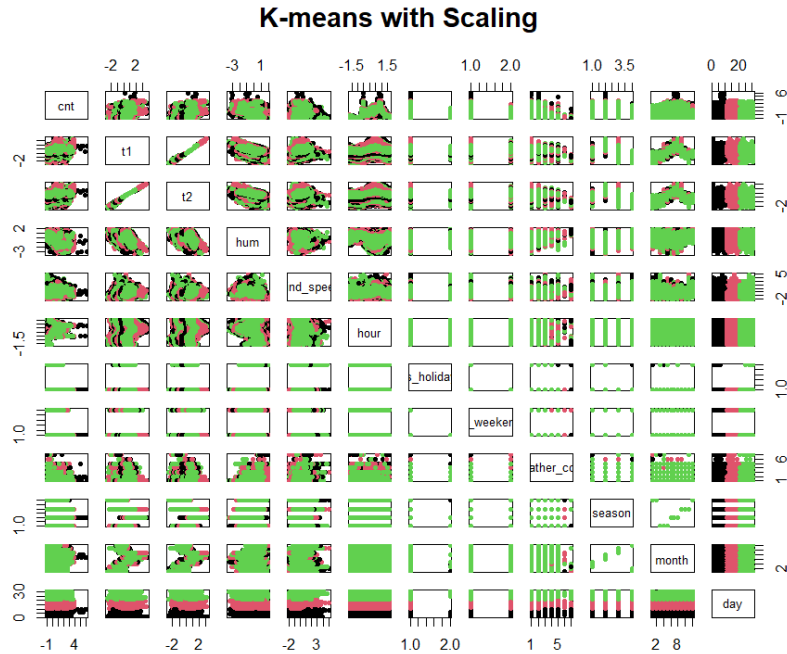
Ultimately, we decided that 3 clusters was the optimal point at which the total within-cluster sum of squares was minimized without needing an overly long computational time that is usually associated with the increase in number of clusters.

```
km.res.3 <- kmeans(train.data.sc, 3, nstart=20)
```

```
km.res.3
```

```
plot(train.data.sc, col = km.res.3$cluster, pch=20, main="K-means with Scaling")
```

Below is a visualization of the 3 clusters created from the training dataset.



Based on the scatterplot matrix above, there is no inherently obvious way to describe the clusters and their relationship with the predictor variables. While the scatterplots with the day predictor variable show clearer separation of observations and the clusters they are group into, the clusters do not provide any meaningful insight, as the clusters are generally just stratifications or groupings of the observations based on the day in the month, without much influence from other predictor variables.

Based on the clustering results, we performed a multinomial logistic regression to develop a model that can potentially predict which cluster each observation in the test data set would fall into. However, a limitation of this model would be that we would be unable to calculate the test MSE of the model as clustering is a form of unsupervised learning, where we do not have predetermined clustering assignments for the test data set to check model accuracy.

```
model <- multinom(cluster ~ ., data = train_data)
summary(model)
```

```

Coefficients:
(Intercept)          cnt          t1          t2          hum wind_speed weather_code2 weather_code3
2  -54.56499 -6.891716e-05 1.019987 -1.489191  0.005787869 -0.2516494    2.492034    1.9728030
3  -54.26932 -1.037551e-03 3.703358 -3.802484 -0.050330676 -0.4065977    3.844482    0.9451038
  weather_code4 weather_code7 weather_code10 weather_code26 is_holiday1 is_weekend1 season1
2    2.509396    3.074056    -4.9244175    0.3134896   -12.860031    0.3444511   -9.899537
3    4.753135    2.800924    -0.5380827   -5.2781952    5.148507    1.2397100   -11.762866
  season2 season3 month2 month3 month4 month5 month6 month7 month8
2  -7.443533 -8.006350 -2.782693 -8.09957 -8.396392 -12.719610  0.2142923 -5.3986927 -4.715136
3  -5.850099 -9.288535 -2.230321 -10.13490 -9.783122 -7.449805 -4.8509831 -0.6194585 -6.292425
  month9 month10 month11 month12 day2 day3 day4 day5 day6 day7
2  -8.415864  2.930652 -1.958320  0.5960562 32.32702 31.56582 28.30294 28.73752 27.22387 27.91074
3  -6.731072 -4.872718  5.753691  2.5057251 25.72063 22.87575 25.09856 23.36376 20.23671 25.77631
  day8 day9 day10 day11 day12 day13 day14 day15 day16 day17
2  25.09287 31.29157 27.50429 94.19522 91.90206 98.90984 99.75133 93.66288 95.55875 94.33980
3  23.78498 26.20212 23.38080 67.74635 66.44030 70.17628 72.98868 67.38218 68.33474 67.94348
  day18 day19 day20 day21 day22 day23 day24 day25 day26 day27
2  94.39585 95.43223 96.33543 66.21128 71.02211 68.45630 62.35279 68.31973 70.06658 74.96556
3  64.69471 64.97403 65.68383 104.11992 98.64974 97.76347 93.96340 96.68030 103.81046 100.70872
  day28 day29 day30 day31 hour
2  67.81569 75.09426 64.01660 116.4870 0.9203593
3  98.83323 99.68779 91.78895 137.8805 1.0242639

Std. Errors:
(Intercept)          cnt          t1          t2          hum wind_speed weather_code2 weather_code3
2  27.88500 0.1206786 60.19306 49.98579 2.793904  8.390375    83.29276    102.66047
3  31.86954 0.1242818 57.13322 50.87129 3.565484  6.859299    74.10362    87.87535
  weather_code4 weather_code7 weather_code10 weather_code26 is_holiday1 is_weekend1 season1
2   84.96035   88.46794   5.94785656   0.32308180 3.992724e-05    80.42276   71.09891
3   86.10814   84.73514   0.03726314   0.04516486 6.767688e+01    83.85279   71.93456
  season2 season3 month2 month3 month4 month5 month6 month7 month8
2  71.12491 70.10628 90.24963 69.51697 67.44759 17.95632 59.85109 104.751454 82.15691 84.97435
3  64.09094 89.78878 89.39270 83.01580 84.58887 77.19196 85.78058  3.244162 64.79312 82.72508
  month10 month11 month12 day2 day3 day4 day5 day6 day7 day8
2  79.87126 53.74322 2.370523 65.41104 69.55354 89.44752 122.8192 13.09726  9.025873  0.3293621
3  61.15527 65.89190 8.531221 94.02605 126.89802 57.98254 137.9374 61.28879 130.074917 11.3992126
  day9 day10 day11 day12 day13 day14 day15 day16 day17 day18
2  74.70174  9.034609 8.055175 69.25294 39.38110 59.27857 34.505637 7.040401 26.6219007 27.043055
3  99.82168 44.451024 5.345032  6.86222 39.11049 59.20025  8.265871 1.034164  0.9929098  1.479272
  day19 day20 day21 day22 day23 day24 day25 day26 day27
2  3.3733001 15.23109 2.688646e-05 1.962991 0.2075435  0.001810571  0.05460947  0.00146809 11.34011
3  0.4606422 10.09771 1.265068e-01 2.338229 9.8082259 12.418340716 57.44216050  0.01624246 10.84150
  day28 day29 day30 day31 hour
2  0.008383742 64.98023  0.4914107 74.23135  8.710787
3  0.408039413 65.92511 90.2255261 74.23135  9.927514

Residual Deviance: 0.006835076
AIC: 224.0068

```

Using this multinomial logistic regression model, we fit the testing dataset to predict the cluster that each observation would be categorized into based on the previous k-means clustering model.

```

predictions <- predict(model, newdata = test_data, type = "probs")
predicted_cluster <- as.factor(apply(predictions, 1, function(x) {
  round(which.max(x))
}))
test_data$predicted_cluster <- predicted_cluster

```

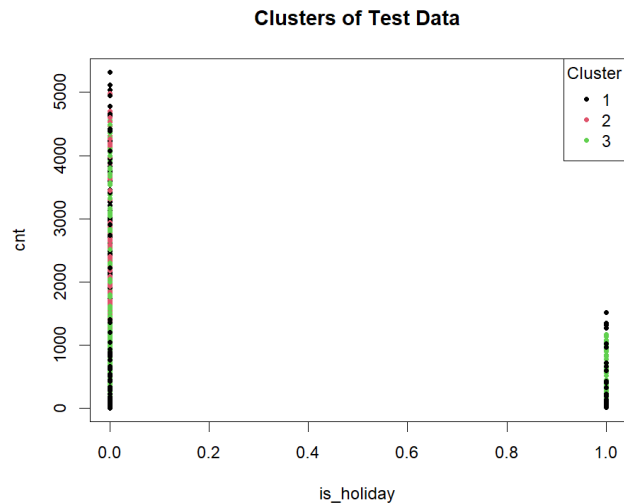
We then plotted the results of the predicted clusters for the test dataset. For example, in our exploratory data analysis, we found that the is_holiday predictor variable would have the largest impact in magnitude on the count of bike shares based on the linear regression. Plotting this relationship in regards to the predicted clusters results in the chart below:

```

plot(test_data$is_holiday, test_data$cnt, col = test_data$predicted_cluster, pch = 20, xlab =
"is_holiday", ylab = "cnt", main = "Clusters of Test Data")

```

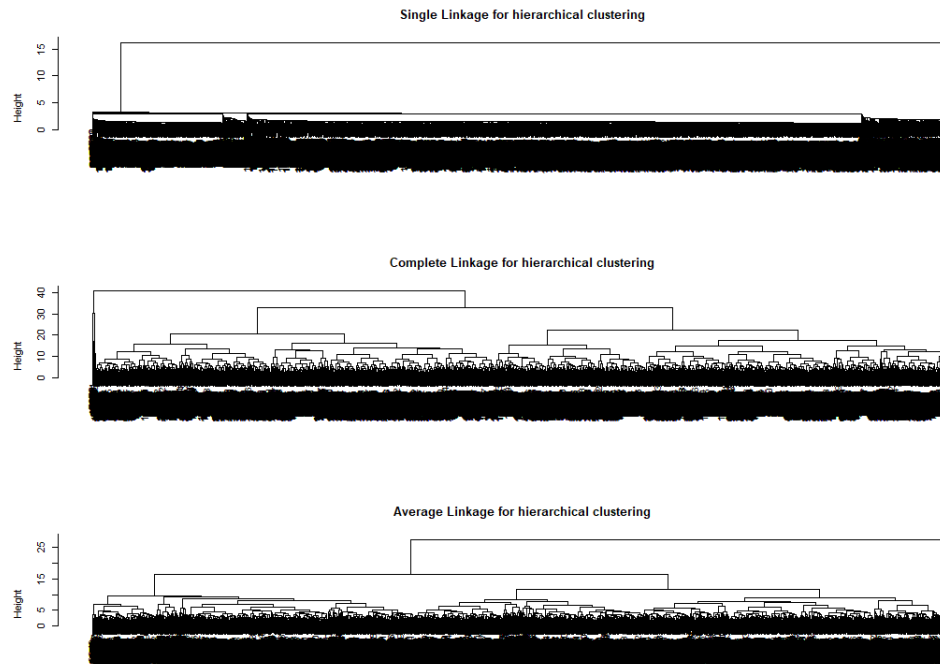
```
legend("topright", legend = unique(test_data$predicted_cluster), col =  
unique(test_data$predicted_cluster), pch = 20, title = "Cluster")
```



However, similar to the results found in the training data set, there are no significant insights or characteristics of the clusters found since there is no strong segmentation between the predicted clusters.

Hierarchical Clustering

In addition to K-means clustering, we also conducted hierarchical clustering to address limitations in the k-means clustering algorithm of finding local optimums. The following dendrograms have been generated based on hierarchical clustering with single linkage, complete linkage, and average linkage.



Based on the dendrograms, it seems that complete linkage and average linkage produce more even and well-balanced clusters than single linkage. We decided to pursue complete linkage as the clusters formed do not seem to merge as early as average linkage, which means that it is less likely for different groups to be merged together at an earlier stage.

Cutting the dendrogram to create three clusters like we did for k-means clustering, we assigned the cluster values to each observation and ran a multinomial logistic regression to develop a model to predict the clusters for the test dataset.

```
train_data$cluster <- factor(cutree(train.complete, 3))
```

```
model <- multinom(cluster ~ ., data = train_data)
```

```
summary(model)
```

```

> summary(model)
Call:
multinom(formula = cluster ~ ., data = train_data)

Coefficients:
(Intercept)          cnt          t1          t2          hum wind_speed weather_code2 weather_code3 weather_code4 weather_code7 weather_code10 weather_code26 is_holiday1
2  -21.29051  0.0005661074 -0.8655330  0.6496798 -0.022193837  0.24312532  -0.1302430  -0.5107357  -1.66935  1.820841  -2.281210  82.23760  -7.876419
3  -21.05294  -0.0001742806 -0.8503092  0.8115304 -0.008936083 -0.01984406  0.2525923  -0.5783016  -1.18508  1.008833  -1.624554  22.61603  -1.810375
   is_weekend1 season1 season2 season3 month2 month3 month4 month5 month6 month7 month8 month9 month10 month11 month12 day2
2  0.1626930 -3.674891 -2.465136 -2.551939 1.3462642 -6.072430 -5.086967 -1.439150 5.1481125 0.3688711 -9.1918749 0.708439 3.319443 -6.493018 1.010074 3.5976022
3  -0.1369971 -2.200016 -6.114361 -4.814103 0.8917343 -1.938741 -2.153627 -3.832096 -0.9040084 -0.4418125 -0.8541947 -0.895180 -1.203311 -4.015870 -0.598232 -0.1840725
   day3 day4 day5 day6 day7 day8 day9 day10 day11 day12 day13 day14 day15 day16 day17 day18 day19 day20
2  7.035660 8.182910 3.510591 2.811580 0.1140924 -0.1197964 4.104806 5.027071 3.216029 6.678048 5.501422 -1.230523 6.252938 6.655299 2.810637 11.99479 11.44733 27.99654
3  6.185747 6.101705 9.295739 6.510101 4.0166152 9.7692949 4.135177 1.652294 9.234239 11.116343 -3.750128 -14.453701 21.502900 25.292868 27.499194 31.25434 34.63780 54.00126
   day21 day22 day23 day24 day25 day26 day27 day28 day29 day30 day31 hour
2  29.53796 28.28674 21.94648 26.40483 29.02052 25.93173 25.28089 25.49050 20.99397 26.20098 6.262062 0.07049353
3  51.95204 51.67768 52.59659 51.44838 51.00971 49.25784 46.63550 51.30198 48.16119 49.46987 46.211075 0.03091588

Std. Errors:
(Intercept)          cnt          t1          t2          hum wind_speed weather_code2 weather_code3 weather_code4 weather_code7 weather_code10 weather_code26 is_holiday1
2  0.002108865 5.843764e-03 0.02541287 0.0244311 0.166603181 0.06279021 0.0002531694 0.0007546034 0.049503e-05 0.00109149 3.300490e-08 6.327143e-12 6.973681e-08
3  0.046332464 8.665809e-05 0.12072012 0.1008528 0.003942691 0.01505022 0.1947033053 0.1949937428 1.746306e-01 0.25553524 4.111194e-04 6.327140e-12 2.969214e-08
   is_weekend1 season1 season2 season3 month2 month3 month4 month5 month6 month7 month8 month9 month10 month11 month12
2  0.0006420361 0.0001004882 0.0005697622 0.001193719 0.0002510186 9.309964e-07 5.268005e-07 0.000254065 9.633143e-05 4.179347e-06 4.550896e-10 6.002093e-05 0.000458133
3  0.1732270555 0.1552351628 0.1623694549 0.161802855 0.2016243324 1.764913e-01 2.167663e-01 0.202558146 2.025482e-01 1.997223e-01 2.078365e-01 2.324564e-01 0.229344649
   month11 month12 day2 day3 day4 day5 day6 day7 day8 day9 day10 day11 day12
2  5.331451e-05 0.000666765 1.638277e-04 2.148738e-04 4.448840e-04 4.685237e-05 4.405151e-05 3.207182e-07 2.498811e-06 1.638190e-05 2.469005e-05 1.789694e-07 8.345588e-05
3  1.525182e-01 0.181469062 1.730449e-11 1.062459e-08 9.072495e-09 2.812900e-07 1.339562e-08 1.450817e-09 4.478676e-07 1.310231e-09 1.309654e-10 3.413515e-07 2.024358e-06
   day13 day14 day15 day16 day17 day18 day19 day20 day21 day22 day23 day24 day25
2  4.524771e-05 1.117014e-07 8.282549e-05 3.484546e-05 1.253417e-07 0.0005125986 1.263104e-05 8.831761e-07 0.0001422849 7.833122e-05 1.356546e-08 7.765384e-06 3.044572e-05
3  6.499500e-13 1.478001e-17 1.818862e-01 1.601660e-01 1.584230e-01 0.2300627099 1.903330e-01 8.833931e-07 0.0001422931 7.833230e-05 1.800152e-08 7.767895e-06 3.046001e-05
   day26 day27 day28 day29 day30 day31
2  2.689518e-05 0.0001130134 4.435784e-06 9.997648e-07 2.283760e-05 7.367885e-12 0.02775166
3  2.690685e-05 0.0001132572 4.453932e-06 1.041570e-06 2.285524e-05 2.501315e-07 0.01251585

Residual Deviance: 1061.002
AIC: 1285.002

```

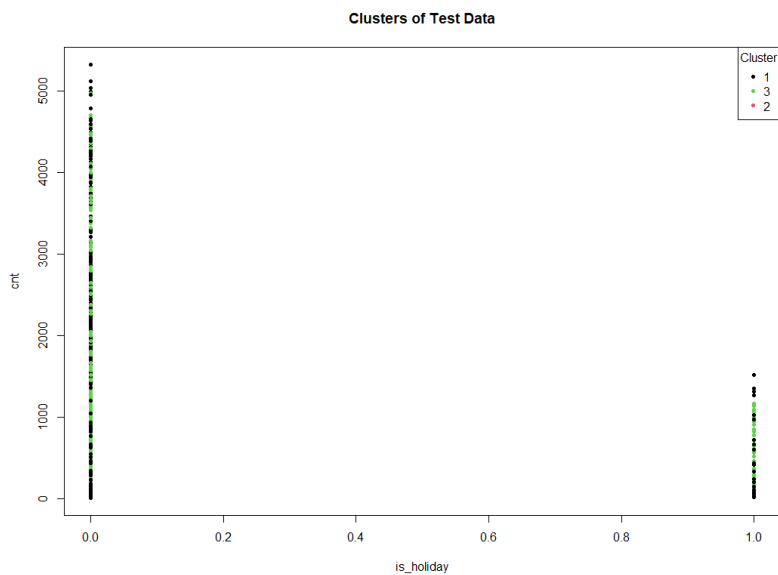
Using this model, we predicted the clusters for each observation in the test data set.

```

predictions <- predict(model, newdata = test_data, type = "probs")
predicted_cluster <- as.factor(apply(predictions, 1, function(x) {
  round(which.max(x))
}))
test_data$predicted_cluster <- predicted_cluster

```

Plotting the clusters in relation to count of bikeshare and is_holiday, we render the following plot:



Following a similar conclusion from our k-means clustering, the hierarchical clustering does not provide significant insight into the trends among observations and the predictor variables. Thus, it is difficult to adapt a clustering model to predict the count of bike shares at a given time.

Linear Regression

We created a Linear Regression Model of the count of bike shares (“cnt”) on all the predictor values.

Before fitting the model, we converted all categorical variables – “isholiday”, “isweekend”, “weather_code”, and “season” – to factor variables:

```
data$is_holiday <- as.factor(data$is_holiday)
data$is_weekend <- as.factor(data$is_weekend)
data$weather_code <- as.factor(data$weather_code)
data$season <- as.factor(data$season)
```

We also converted timestamp to POSIXct format and extracted month, day, and hour to determine if specific times had an impact on the “cnt”:

```
data$timestamp <- as.POSIXct(data$timestamp, format = "%m/%d/%Y %H:%M")

# Extracting month, day, and hour
data$month <- month(data$timestamp)
data$day <- day(data$timestamp)
data$hour <- hour(data$timestamp)
```

After completing data valuation, we used the *lm()* function to fit a linear model with all predictor variables:

```
lm_1 <- lm(formula = cnt ~., data = data)
summary(lm_1)
```

The result is as follows:

```

> summary(lm_1)

Call:
lm(formula = cnt ~ ., data = data)

Residuals:
    Min       1Q   Median       3Q      Max
-1926.3  -573.2  -244.6   294.9  6032.2

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  1.675e+03  7.095e+02   2.361 0.018258 *
timestamp    2.455e-07  4.968e-07   0.494 0.621156
t1           8.734e+01  1.009e+01   8.660 < 2e-16 ***
t2          -2.961e+01  8.269e+00  -3.581 0.000343 ***
hum         -2.441e+01  7.301e-01 -33.434 < 2e-16 ***
wind_speed  -7.926e+00  1.082e+00  -7.324 2.52e-13 ***
weather_code2  2.839e+02  1.971e+01  14.401 < 2e-16 ***
weather_code3  2.892e+02  2.172e+01  13.314 < 2e-16 ***
weather_code4 -1.554e+02  3.069e+01  -5.064 4.14e-07 ***
weather_code7 -1.523e+01  2.684e+01  -0.567 0.570508
weather_code10 -4.710e+02  2.382e+02  -1.977 0.048050 *
weather_code26  1.206e+02  1.265e+02   0.954 0.340336
is_holiday1  -2.200e+02  5.182e+01  -4.246 2.19e-05 ***
is_weekend1  -1.899e+02  1.617e+01 -11.741 < 2e-16 ***
season1      -5.629e+01  2.617e+01  -2.150 0.031537 *
season2       1.642e+02  2.985e+01   5.501 3.84e-08 ***
season3       2.283e+01  2.158e+01   1.058 0.290188
month        -2.187e+01  3.680e+00  -5.942 2.88e-09 ***
day          -3.551e+00  8.291e-01  -4.283 1.86e-05 ***
hour         2.889e+01  1.122e+00  25.757 < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 889.3 on 15114 degrees of freedom
Multiple R-squared:  0.3443,    Adjusted R-squared:  0.3435
F-statistic: 417.7 on 19 and 15114 DF,  p-value: < 2.2e-16

```

In this model, we see that “*timestamp*”, “*weather_code7*”, “*weather_code26*”, and “*season3*” are not statistically significant, and we can therefore assume that they do not have a tangible influence on “*cnt*”. It is also important to note that the Multiple R-squared value is only 0.3443, which means that only 34.43% of the proportion of the variance in the response variable can be explained by our predictor variables and is not the best fit for this data. In this model, “*weather_code10*” has the largest estimated coefficient at -4.710e+02, indicating that if “*weather_code10*” is true, there is the largest magnitude of impact on the number of bike shares. This coefficient is also negative, meaning that we expect a decrease in “*cnt*”.

If we run *anova()* of this model, we find the following:


```

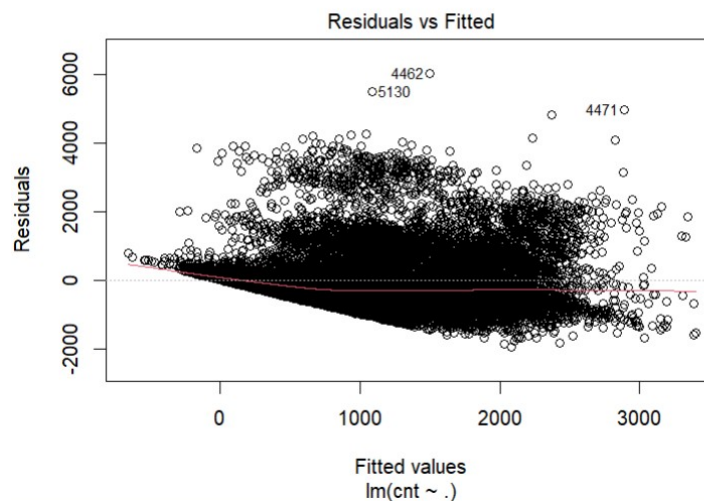
> anova(lm_1)
Analysis of Variance Table

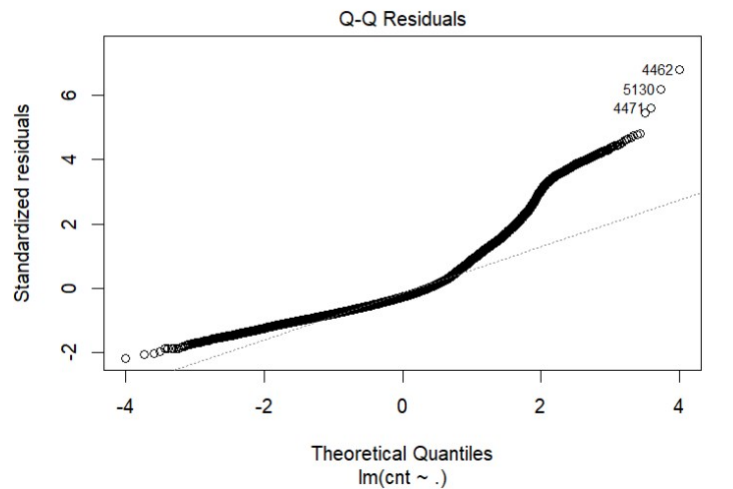
Response: cnt
      Df    Sum Sq   Mean Sq    F value    Pr(>F)
timestamp 1 1.4469e+08 144687855  182.959 < 2.2e-16 ***
t1         1 2.7889e+09 2788867347 3526.547 < 2.2e-16 ***
t2         1 2.3447e+08 234468473  296.487 < 2.2e-16 ***
hum        1 1.8991e+09 1899116889 2401.450 < 2.2e-16 ***
wind_speed 1 6.5749e+06  6574923    8.314 0.0039395 **
weather_code 6 4.3098e+08  71830806  90.831 < 2.2e-16 ***
is_holiday 1 1.0880e+07  10879721   13.758 0.0002087 ***
is_weekend 1 1.0676e+08 106756890  134.995 < 2.2e-16 ***
season     3 7.0718e+07  23572508   29.808 < 2.2e-16 ***
month      1 4.0768e+07  40767882   51.551 7.30e-13 ***
day        1 1.7126e+07  17125862   21.656 3.29e-06 ***
hour       1 5.2464e+08 524642944 663.415 < 2.2e-16 ***
Residuals 15114 1.1952e+10  790821
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
>

```

Here, we can see that “t1”, “hum”, and “hour” have notably high F-values, which tells us that there is a difference in the group means and that these variables might have a larger impact on the response variable “cnt”.

Again, from the Multiple R-squared value, we suspect that a linear model with all predictor values does not fit the data, but we can also test whether the data fits a linear model using *plot()*.

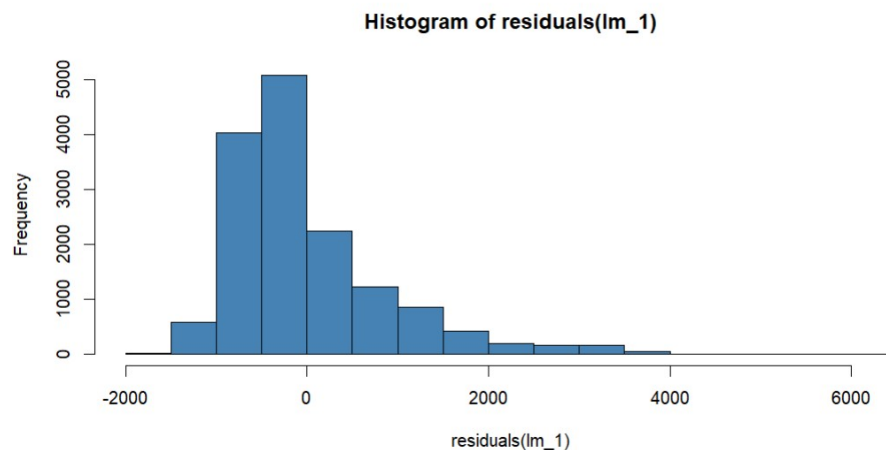




We see in the Residual vs. Fitted plot that the residuals are not completely randomly distributed as they show a line of concentration below the 0 line between values 0-1000. Furthermore, the Q-Q Residuals plot does not follow the vertical line and significantly strays between quantiles 1-4. Therefore, we know that the data does not fit a linear model.

The histogram of `lm_1`'s residuals also show that the data is skewed to the right, so we cannot assume normality:

```
hist(residuals(lm_1), col = "steelblue")
```



We also found a Test MSE of 910,762.7, which is abnormally large and further supports that the linear model is not super accurate.

```

> predictions1 <- predict(lm_1, newdata = test_data)
> test_mse1 <- mean((test_data$cnt - predictions1)^2)
> test_mse1
[1] 910762.7

```

After looking at the linear regression model with all predictor variables, we wanted to create a model with only statistically significant variables. Since “timestamp”, “weather_code7”, “weather_code26”, and “season3” are not statistically significant, we want to remove them from the model. However, because “weather_code7”, “weather_code26”, and “season3” are factor levels of the variables “weather_code” and “season”, we first need to create dummy variables for each level:

```

# Using model.matrix to segregate factor levels
season_matrix <- model.matrix(~ 0 + season, data = data)
weather_matrix <- model.matrix(~ 0 + weather_code, data = data)

# Combining the new distributed variables to data and deducting original variables
final_data <- cbind(data, season_matrix, weather_matrix)

# Omit one level from dummy variables to avoid multicollinearity
final_data <- dplyr::select(final_data, -weather_code, -weather_code1, -season, -season0)

```

After going through this process, we can create a linear model with all statistically significant variables:

```

lm_3 <- lm(formula = cnt ~ . - timestamp - season3 - weather_code7 - weather_code26, data =
final_data)
summary(lm_3)

```

```

Call:
lm(formula = cnt ~ . - timestamp - season3 - weather_code7 -
    weather_code26, data = final_data)

Residuals:
    Min       1Q   Median       3Q      Max
-1927.3  -573.9  -244.8   297.9   6026.6

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  2036.9918    72.9860   27.909 < 2e-16 ***
t1             88.3248     9.9979    8.834 < 2e-16 ***
t2            -30.8974     8.1994   -3.768 0.000165 ***
hum            -24.4255     0.6493  -37.620 < 2e-16 ***
wind_speed     -7.9740     1.0372   -7.688 1.58e-14 ***
is_holiday1   -223.4493    51.7222   -4.320 1.57e-05 ***
is_weekend1   -189.7988    16.1671  -11.740 < 2e-16 ***
month         -21.5470     3.6563   -5.893 3.87e-09 ***
day           -3.5413     0.8275   -4.280 1.88e-05 ***
hour           28.9159     1.1141   25.954 < 2e-16 ***
season1       -60.5690    25.5795   -2.368 0.017903 *
season2       156.7404    28.7232    5.457 4.92e-08 ***
weather_code2  288.1271    18.4621   15.606 < 2e-16 ***
weather_code3  294.1043    19.5511   15.043 < 2e-16 ***
weather_code4 -152.1351    29.3870   -5.177 2.28e-07 ***
weather_code10 -469.7749    237.9845   -1.974 0.048403 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 889.2 on 15118 degrees of freedom
Multiple R-squared:  0.3442,    Adjusted R-squared:  0.3435
F-statistic: 528.9 on 15 and 15118 DF,  p-value: < 2.2e-16

```

Here, we see that while all the predictor variables are significant, the Multiple R-Squared is virtually the same at 0.3442. Like the first model, the estimated coefficient of “*weather_code10*” has the largest magnitude of impact on the response variable at -469.7749. “*weather_code2*” and “*weather_code3*” have the next largest magnitudes of impact in this model at 288.1271 and 294.1043 respectively, so we can derive that “*weather_code*” overall seems to have a significant impact on “*cnt*”. Since “*weather_code2*” and “*weather_code3*” represent scattered and broken clouds, we can assume that the positive relationship with the number of bike shares results from the nicer weather, which creates good bike riding conditions. Alternatively, “*weather_code10*” represents rain with thunderstorms which creates unpleasant and dangerous riding conditions, which deters people from biking.

If we run `anova()`, we find the following:

```

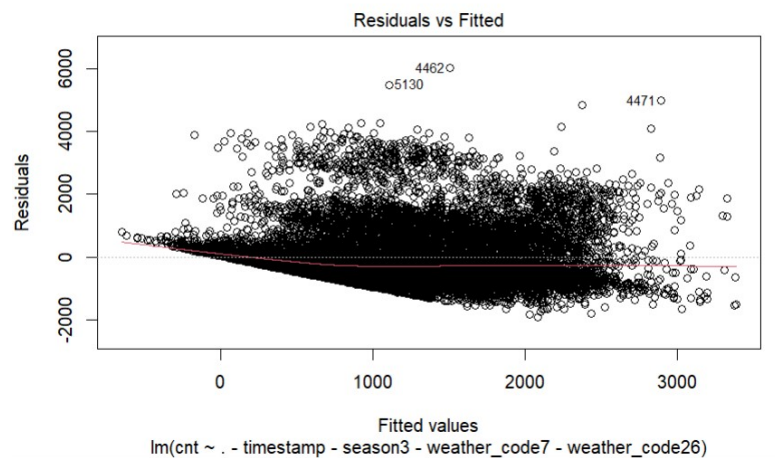
> anova(lm_3)
Analysis of Variance Table

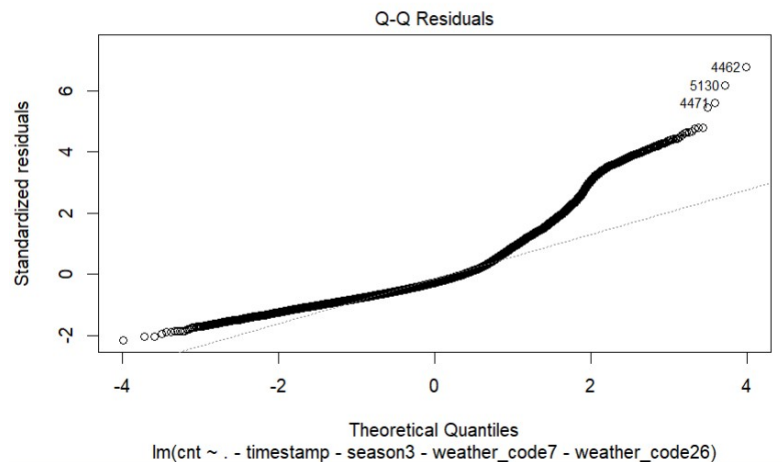
Response: cnt
      Df    Sum Sq   Mean Sq    F value    Pr(>F)
t1      1 2.8889e+09 2888897532 3653.3346 < 2.2e-16 ***
t2      1 2.4205e+08 242051962  306.1018 < 2.2e-16 ***
hum      1 1.9361e+09 1936075721 2448.3846 < 2.2e-16 ***
wind_speed 1 6.6738e+06   6673771    8.4397 0.0036764 **
is_holiday 1 1.0327e+07  10327274   13.0600 0.0003026 ***
is_weekend 1 1.2240e+08 122401284  154.7901 < 2.2e-16 ***
month     1 7.2341e+06   7234092    9.1483 0.0024936 **
day       1 1.6423e+07  16422641   20.7683 5.224e-06 ***
hour      1 6.0378e+08  603784287  763.5529 < 2.2e-16 ***
season1    1 3.7002e+07  37002022   46.7932 8.191e-12 ***
season2    1 3.0496e+07  30495635   38.5651 5.434e-10 ***
weather_code2 1 1.3093e+08 130933581 165.5802 < 2.2e-16 ***
weather_code3 1 2.1707e+08 217074097 274.5145 < 2.2e-16 ***
weather_code4 1 2.0951e+07  20950957   26.4948 2.676e-07 ***
weather_code10 1 3.0812e+06   3081236    3.8966 0.0484030 *
Residuals 15118 1.1955e+10   790756
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
>

```

Again, we see that “*t1*”, “*hum*”, and “*hour*” have very high F values. In fact, the F value of “*t1*” increases from 3526.547 to 3653.3346; the F value of “*hum*” increases from 2401.450 to 2448.3846; and the F value of “*hour*” increases from 663.415 to 763.5802. These increases suggest that a greater proportion of the variance in the response variable is explained by the explanatory variables and that the model fit and predictive power is potentially better than the model with all predictor variables.

If we look at the Residuals vs. Fitted and Q-Q Residuals plot, however, we see very similar patterns as the first linear model with all predictor variables. Again, this model does not appear to fit a linear model.





Overall, the linear model with all statistically significant variables show nearly identical patterns in their Residual plots, and we cannot assume that this model will be super accurate, as it does not appear to fit linear regression.

If we look at the calculated Test MSE, we actually see a slight increase from the first model to 919,153.8. Again, this Test MSE is abnormally high and indicates that this model does not fit the data well.

```
> predictions3 <- predict(lm_3, newdata = test_final_data)
> test_mse3 <- mean((test_data$cnt - predictions3)^2)
> test_mse3
[1] 919153.8
```

Since our models did not fit the data well, we wanted to explore multicollinearity, which is one of the assumptions of the linear regression model. We already see that the data is not normalized from the Residual Histogram, Residual vs. Fitted plots, and the Q-Q plots. The correlation matrix model below also indicates that some predictor variables have very high correlations, and therefore, this data does not have the multicollinearity that we expect when creating a linear model. “t1” and “t2” unsurprisingly have a very high correlation at 0.99, and variables such as “season2”, “season1”, and “month” also have high correlations with other variables, which further explains why the model had such low Multiple R-squared values and high Test MSE.

```
## Multicollinearity assumption check ##
```

```
install.packages("ggcorrplot")
```

```
library(ggcorrplot)
```

```
#Create data copy to modify
```

```
data_copy <- data
```

```
# Convert categorical data to numeric
```

```
data_copy$is_holiday <- as.numeric(data_copy$is_holiday)
```

```
data_copy$is_weekend <- as.numeric(data_copy$is_weekend)
```

```
season_encoded <- model.matrix(~ season - 1, data = data_copy)
```

```
season_numeric <- apply(season_encoded, 2, as.numeric)
```

```
weather_encoded <- model.matrix(~ weather_code - 1, data = data_copy)
```

```
weather_numeric <- apply(weather_encoded, 2, as.numeric)
```

```
data_copy <- cbind(data, season_numeric, weather_numeric)
```

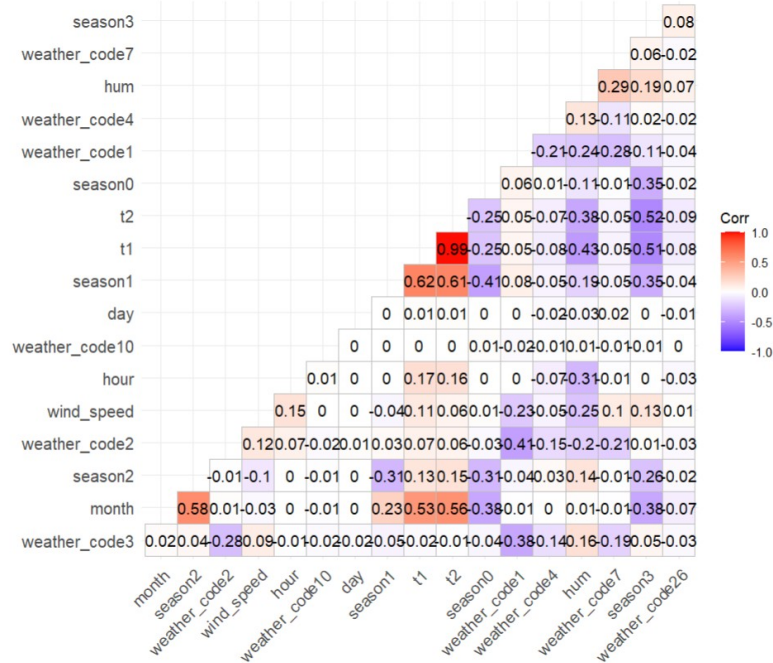
```
data_copy <- na.omit(data_copy)
```

```
numeric_data <- subset(data_copy, select = -c(cnt, timestamp, weather_code, is_holiday,  
      is_weekend, season))
```

```
numeric_corr_matrix = round(cor(numeric_data), 2)
```

```
numeric_corr_matrix
```

```
ggcorrplot(numeric_corr_matrix, hc.order = TRUE, type = "lower", lab = TRUE)
```



Polynomial Regression Model

After trying linear regression and finding that it might not be the best model for this dataset we used polynomial terms to model non-linear relationships between the predictors and the dependent variable. Before fitting the model, we did similar steps as shown in the previous regression model. We converted all categorical variables – “isholiday”, “isweekend”, “weather_code”, and “season” – to factor variables; converted timestamp to POSIXct format to month, day, and how to better predict ‘cnt’(same for testing_data):

```
data$isholiday <- as.factor(data$isholiday)
data$isweekend <- as.factor(data$isweekend)
data$weather_code <- as.factor(data$weather_code)
data$season <- as.factor(data$season)

# Convert timestamp to POSIXct format
data$timestamp <- as.POSIXct(data$timestamp, format = "%m/%d/%Y %H:%M")
data <- na.omit(data)

# Extract month, day, and hour
data$month <- month(data$timestamp)
```



```
data$day <- day(data$timestamp)
data$hour <- hour(data$timestamp)
```

First, we generated polynomial terms for continuous variables like temperature, humidity, and wind speed.

```
### Polynomial Model ###
# Generate polynomial terms for continuous variables to the 2nd degree
data$t1_poly <- poly(data$t1, 2, raw=TRUE)
data$hum_poly <- poly(data$hum, 2, raw=TRUE)
data$wind_speed_poly <- poly(data$wind_speed, 2, raw=TRUE)

# View structure of the new polynomial variables
str(data[c("t1_poly", "hum_poly", "wind_speed_poly")])

# Fit a polynomial regression model
polynomial_model <- lm(cnt ~ t1_poly + t2 + hum_poly + wind_speed_poly + weather_code +
season + hour , data = data)
summary(polynomial_model)
```

```

Call:
lm(formula = cnt ~ t1_poly + t2 + hum_poly + wind_speed_poly +
    weather_code + season + hour, data = data)

Residuals:
    Min       1Q   Median       3Q      Max
-2495.3  -546.8  -245.3   235.4  6110.9

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)   3177.71407   154.20214   20.607 < 2e-16 ***
t1_poly1      -57.95326    14.60119   -3.969 7.25e-05 ***
t1_poly2       2.77141     0.21830    12.695 < 2e-16 ***
t2            23.52701     9.12301     2.579 0.00992 **
hum_poly1     -55.30600     4.40846  -12.545 < 2e-16 ***
hum_poly2       0.23372     0.03232     7.232 5.00e-13 ***
wind_speed_poly1 23.12347     3.52481     6.560 5.55e-11 ***
wind_speed_poly2 -0.67350     0.08448    -7.972 1.67e-15 ***
weather_code2   317.15388    19.75856    16.051 < 2e-16 ***
weather_code3   306.40103    21.67208    14.138 < 2e-16 ***
weather_code4  -162.69069    30.53360    -5.328 1.01e-07 ***
weather_code7   -15.44560     27.18119    -0.568 0.56988
weather_code10 -412.07307    237.50462    -1.735 0.08276 .
weather_code26  -19.10872    126.43294    -0.151 0.87987
season1        -83.92225    25.71416    -3.264 0.00110 **
season2         95.86306    24.03689     3.988 6.69e-05 ***
season3        -37.45426    22.12021    -1.693 0.09044 .
hour           29.17961     1.11871    26.083 < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 886.4 on 15116 degrees of freedom
(2 observations deleted due to missingness)
Multiple R-squared:  0.3484,    Adjusted R-squared:  0.3477
F-statistic: 475.4 on 17 and 15116 DF,  p-value: < 2.2e-16

```

The multiple R-squared is 0.3484, which is still pretty low and similar to linear regression. The multiple R-squared value also indicates that approximately 34.84% of the variability in 'cnt' is explained by this model.

Trying more degrees:

```
degree <- 3
```

```

test_model2 <- lm(cnt ~ poly(t1, degree) + t2 + poly(hum, degree)
    + poly(wind_speed, degree) + weather_code
    + season + hour, data = data)
summary(test_model2)

```

```

Call:
lm(formula = cnt ~ poly(t1, degree) + t2 + poly(hum, degree) +
    poly(wind_speed, degree) + weather_code + season + hour,
    data = data)

Residuals:
    Min       1Q   Median       3Q      Max
-1866.9  -548.3  -242.8   237.7  6128.3

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)      624.191    117.143   5.328 1.00e-07 ***
poly(t1, degree)1 23525.697    8116.829   2.898 0.00376 **
poly(t1, degree)2 13756.549    1111.911  12.372 < 2e-16 ***
poly(t1, degree)3 -5553.626     962.144  -5.772 7.98e-09 ***
t2                10.130       9.436   1.074 0.28304
poly(hum, degree)1 -41740.244    1274.940 -32.739 < 2e-16 ***
poly(hum, degree)2  7379.578     957.642   7.706 1.38e-14 ***
poly(hum, degree)3  3797.435     908.871   4.178 2.95e-05 ***
poly(wind_speed, degree)1 -2294.392    1105.482  -2.075 0.03796 *
poly(wind_speed, degree)2 -7467.030     928.515  -8.042 9.50e-16 ***
poly(wind_speed, degree)3 -137.632     899.325  -0.153 0.87837
weather_code2       301.386       19.970  15.092 < 2e-16 ***
weather_code3       299.844       21.671  13.836 < 2e-16 ***
weather_code4      -164.044       30.518  -5.375 7.76e-08 ***
weather_code7       -23.401       27.230  -0.859 0.39014
weather_code10     -430.325      237.110  -1.815 0.06956 .
weather_code26     -93.255      126.610  -0.737 0.46141
season1            -131.612       26.796  -4.912 9.13e-07 ***
season2              74.000       24.245   3.052 0.00228 **
season3            -42.777       22.190  -1.928 0.05390 .
hour               28.810        1.119  25.755 < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 884.9 on 15113 degrees of freedom
(2 observations deleted due to missingness)
Multiple R-squared:  0.3508,    Adjusted R-squared:  0.35
F-statistic: 408.3 on 20 and 15113 DF,  p-value: < 2.2e-16

```

The multiple R-squared is 0.3508, which is still pretty low and similar to linear regression. This also indicates that approximately 35.8% of the variability in 'cnt' is explained by this model. In general it didn't increase that much.

Since this polynomial model had the highest R-squared value among the polynomial regression models we built, we also calculated the Test MSE for this model. While these Test MSE is slightly lower than the Test MSE for the Linear Models, which were in the 900,000s, the value of 849,782.2 is still abnormally high and further suggests that this model is not accurate for the data.

```

> prediction <- predict(model_test, newdata = test_data)
>
> test_mse <- mean((test_data$cnt - prediction)^2)
> test_mse
[1] 849782.2
> |

```

Trying more degrees:

```
degree <- 4
```

```
test_model3 <- lm(cnt ~ poly(t1, degree) + t2 + poly(hum, degree)
```

```

+ poly(wind_speed, degree) + weather_code
+ season + hour, data = data)

summary(test_model3)

Call:
lm(formula = cnt ~ poly(t1, degree) + t2 + poly(hum, degree) +
    poly(wind_speed, degree) + season + hour, data = data)

Residuals:
    Min       1Q   Median       3Q      Max
-1910.0   -547.3   -263.1    235.0   6296.2

Coefficients:
                Estimate Std. Error t value Pr(>|t|)
(Intercept)      315.999    131.388   2.405  0.0162 *
poly(t1, degree)1 -3307.916   9132.334  -0.362  0.7172
poly(t1, degree)2 14914.611   1155.748  12.905 < 2e-16 ***
poly(t1, degree)3 -5471.386    980.118  -5.582 2.41e-08 ***
poly(t1, degree)4 -7057.292   1029.807  -6.853 7.51e-12 ***
t2                44.466     10.697   4.157 3.24e-05 ***
poly(hum, degree)1 -42385.845   1149.793 -36.864 < 2e-16 ***
poly(hum, degree)2  5161.236    950.034   5.433 5.64e-08 ***
poly(hum, degree)3  4880.042    913.175   5.344 9.22e-08 ***
poly(hum, degree)4  127.567    901.990   0.141  0.8875
poly(wind_speed, degree)1 1764.917   1102.678   1.601  0.1095
poly(wind_speed, degree)2 -8674.018    942.500  -9.203 < 2e-16 ***
poly(wind_speed, degree)3  319.970    913.809   0.350  0.7262
poly(wind_speed, degree)4 1586.386    902.417   1.758  0.0788 .
season1           -127.675     27.183  -4.697 2.67e-06 ***
season2            111.004     24.712   4.492 7.11e-06 ***
season3             -9.107     22.472  -0.405  0.6853
hour               28.597      1.132  25.254 < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 898.2 on 15116 degrees of freedom
(2 observations deleted due to missingness)
Multiple R-squared:  0.3309,    Adjusted R-squared:  0.3302
F-statistic: 439.8 on 17 and 15116 DF,  p-value: < 2.2e-16

```

The multiple R-squared is 0.3309, which is still pretty low and similar to other degrees. After adjusting the degree to a bigger number until the 24th degree, the R-squared value is still pretty low. Thus, the polynomial regression model is not an accurate way to predict the specific number of 'cnt'.

Decision Tree

We created a decision tree model of the count of new bike shares (i.e., "cnt") on all the predictor variables.

```

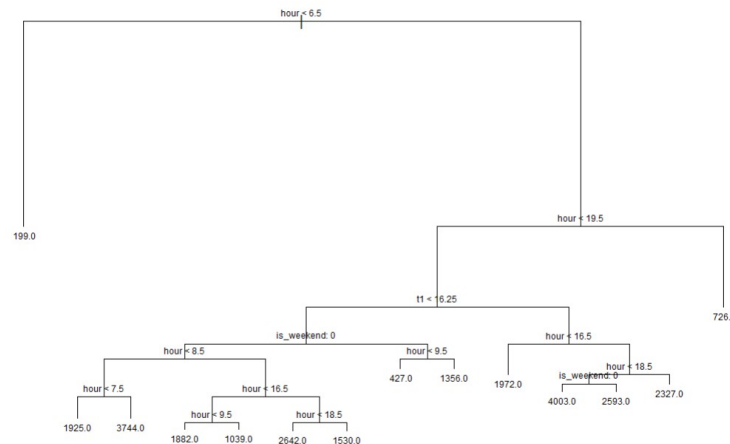
cnt.tree <- tree(cnt~., data = train.df)
summary(cnt.tree)

```

Usually, R's default tree should have used all the predictor variables in the tree construction process. However, there are only three out of ten variables that are used in tree construction: "hour", "t1", and

“is_weekend”. We suspect it is due to two main reasons: (1) our dataset is too large and (2) the three variables are the most important in predicting the target variable “cnt”.

```
Regression tree:
tree(formula = cnt ~ ., data = train.df)
Variables actually used in tree construction:
[1] "hour"      "t1"        "is_weekend"
Number of terminal nodes: 14
Residual mean deviance: 291400 = 4.406e+09 / 15120
Distribution of residuals:
      Min.   1st Qu.   Median     Mean   3rd Qu.    Max.
-3679.00  -211.70   -67.89     0.00   218.10   4613.00
```



To compare the prediction performance of the original decision tree across other models, we computed the testMSE of the original tree (i.e., “orgMSE”) which equals 238475.4

```
orgyhat <- predict(cnt.tree, newdata = test.df)
orgMSE <- mean((orgyhat - test.df$cnt)^2)
> orgMSE <- mean((orgyhat - test.df$cnt)^2)
> orgMSE
[1] 238475.4
```

The testMSE seems very large compared to the numbers we usually obtain from class examples or homework assignments due to the data size difference and normalization. The datasets introduced in our class examples and homework assignments are usually only around hundreds of instances and are normalized. In comparison, our training set has 15,137 instances and is not normalized. Therefore, the orgMSE is actually not very large at all, which is normal under these circumstances.

Since R's default decision tree does not use all the predictor variables, we had to manually write a code to force R to consider all the prediction variables in the tree construction process. This tree is named as "largetree", and the nobs are set equal to 15137 because we have 15,137 instances in our training set.

```
largetree <- tree(cnt~., data = data, subset = train, control = tree.control(nobs = 15137,
minsize = 3, mindev = 0.0005))
summary(largetree)
```

```
Regression tree:
tree(formula = cnt ~ ., data = data, subset = train, control = tree.control(nobs = 15137,
minsize = 3, mindev = 5e-04))
Variables actually used in tree construction:
 [1] "hour"          "is_weekend"    "t1"            "is_holiday"    "season"        "weather_code"
 [7] "day"           "month"         "t2"            "hum"           "wind_speed"
Number of terminal nodes: 77
Residual mean deviance: 90390 = 1.361e+09 / 15060
Distribution of residuals:
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
-2767.00 -112.10  -12.29    0.00  130.00  3669.00
```

The fully grown tree used all the predictor variables in tree construction, but the number of terminal nodes increased from 14 to 77 due to the increase in complexity and dimensionality. Similarly, we computed the testMSE of the fully grown tree (i.e., largeMSE) which equals 132841.6

```
largeyhat <- predict(largetree, newdata = test.df)
largeMSE <- mean((largeyhat - test.df$cnt)^2)
> largeMSE <- mean((largeyhat - test.df$cnt)^2)
> largeMSE
[1] 132841.6
```

The largeMSE is smaller than orgMSE ($132841.6 < 238475.4$). This indicates that the original tree actually under-fitted the data, while the fully grown tree better captures the relationships in the data.

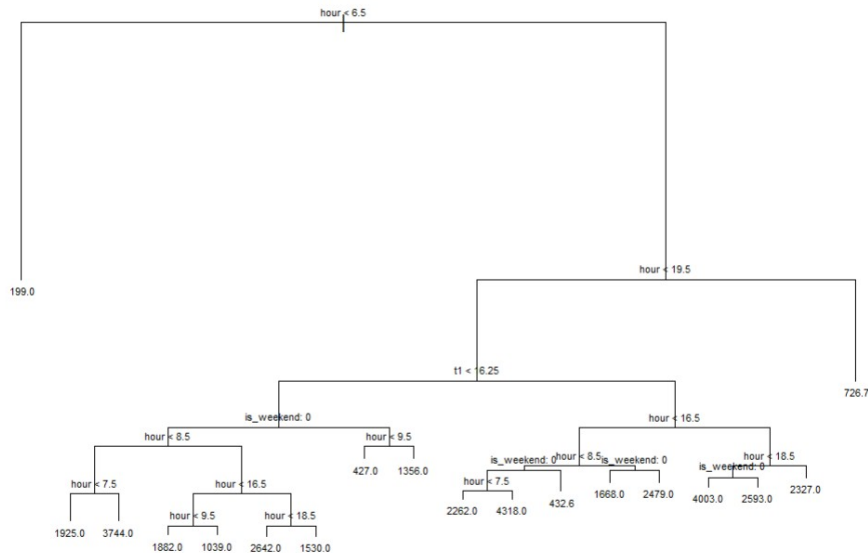
As an attempt to improve the tree model's performance, we tried pruning the tree. First, we found the point of minimum cross-validation error. According to cross-validation, the best pruned tree should have 18 terminal nodes.

```
tree.pruned = prune.tree(largetree, best=18)
summary(tree.pruned)
```

```

Regression tree:
snip.tree(tree = targetree, nodes = c(199L, 209L, 50L, 55L, 196L,
96L, 109L, 108L, 197L, 2L, 198L, 106L, 51L, 7L, 97L, 107L))
Variables actually used in tree construction:
[1] "hour"      "t1"      "is_weekend"
Number of terminal nodes: 18
Residual mean deviance: 225100 = 3.403e+09 / 15120
Distribution of residuals:
      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
-3787.00 -183.00  -58.75   0.00  214.60  4365.00

```



The pruned tree also only used “hour”, “t1”, and “is_weekend” in the tree construction process, which further indicates that these three variables are the most important variables to determine the count of new bike shares. Likewise, we computed the testMSE of the pruned tree (i.e., “prunedMSE”) which equals 1727608.

```

prunedyhat = predict(tree.pruned, newdata = train.df)
prunedMSE <- mean((prunedyhat - test.df$cnt)^2)
> prunedMSE
[1] 1727608

```

The prunedMSE is larger than the largeMSE ($1727608 > 132841.6$). This suggests that the fully grown tree better captures the relationship of predictor variables and count of new bike shares than the pruned tree with the “best” number of terminal nodes. Usually with a fully grown tree, we would worry that it would overfit the prediction model to the training set. However, in this case, it seems like the fully grown tree also underfit the data. Therefore, by pruning the tree, we actually cause the model to lose a lot of the

valuable findings obtained in the fully grown tree. In summary, the pruned tree performs worse than the fully grown tree.

The second method we tried to improve our tree model performance is bagging. We set `mtry` equal to 11, as the bagging method requires `mtry` to be greater or equal to the number of predictor variables.

```
bag <- randomForest(cnt~., data= data, subset = train, mtry=11,importance = TRUE)
```

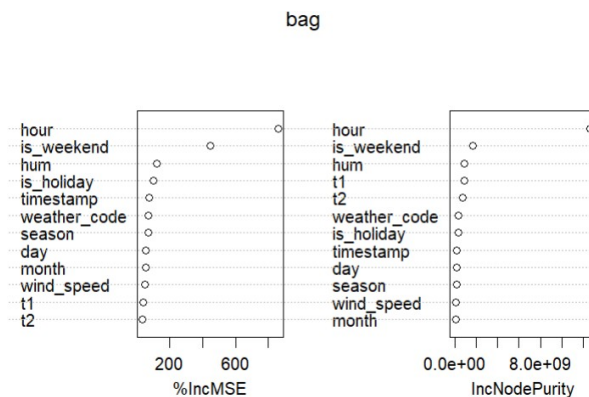
We noticed that the bagging method required a long time to run, so we suspect that it is due to the large size of our training set. We also computed the testMSE of the bagged tree (i.e., `bagMSE`)

```
bagyhat <- predict(bag, newdata = test.df)
bagMSE <- mean((yhat.bag - test.df$cnt)^2)
> bagyhat <- predict(bag, newdata = test.df)
> bagMSE <- mean((bagyhat - test.df$cnt)^2)
> bagMSE
[1] 82640.95
```

The `bagMSE` is smaller than the `largeMSE` ($82640.95 < 132841.6$). We assumed this would be the case as decision trees often suffer from high variance, but bagging reduces the variance. So far, bagging is the best tree model for new bike count prediction.

With bagging, we can create a variable importance plot to indicate the important predictor.

```
importance(bag)
varImpPlot(bag)
```



It seems like “hour” and “is_weekend” are the two most important variables in bagging.

The final method we used to improve our tree model performance is random forest. This time, we set `mtry` equal to 5, as the random forest method requires `mtry` to be less than the number of predictor variables.

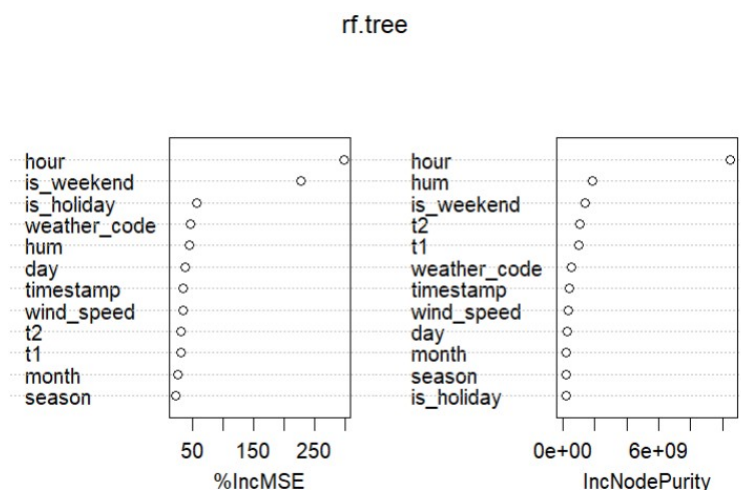
```
rf.tree <- randomForest(cnt ~ ., data = data, subset = train, mtry=5, importance = TRUE)
```

We noticed that the random forest method also took a long time to run, and we suspect it is also because the training dataset is too large. After the previous code finished running, we computed the testMSE of the random forest method (i.e., `rfMSE`) which equals 124954.5.

```
rfyhat <- predict(rf.tree, newdata = test.df)
rfMSE <- mean((rfyhat - test.df$cnt)^2)
> rfyhat <- predict(rf.tree, newdata = test.df)
> rfMSE <- mean((rfyhat - test.df$cnt)^2)
> rfMSE
[1] 124954.5
```

Surprisingly, the `rfMSE` is larger than the `bagMSE` ($124954.5 > 82640.95$). We conclude that this is similar to the case of the fully grown tree and the pruned tree. The bagging method did not result in overfitting the data, as it is under-fitted. Since a random forest is built on a subset of features and a subset of data, it actually causes the model to lose some of the valuable findings obtained in bagging. When we computed the variable importance plot for the random forest model, “hour” and “is_weekend” are still the two most important variables.

```
importance(rf.tree)
varImpPlot(rf.tree)
```



Generalizing all our findings from the previous tree modeling attempts, we can conclude that “hour”, “is_weekend”, and “is_holiday” are the three most important variables. In terms of tree prediction performance comparison, bagging was the best method with the smallest testMSE. Ranking the testMSEs in ascending order: bagMSE < rfMSE < largeMSE < prunedMSE < orgMSE. We noticed that bagging and random forest took a long time to run due to the large size of our dataset, so we will only use “hour”, “is_weekend”, “is_holiday” this time for bagging and random forest. This time, we created a bagging tree of new bike share counts (“cnt”) on “is_holiday”, “is_weekend”, and “hour”. We set mtry equal to 3 because we are only using three predictor variables.

```
bagtree3 <- randomForest(cnt~ is_holiday + is_weekend + hour, data= data, subset = train,
mtry=3,importance = TRUE)
yhatbag3 <- predict(bagtree3, newdata = test.df)
bagMSE3 <- mean((yhatbag3 - test.df$cnt)^2)
bagMSE3

> yhatbag3 <- predict(bagtree3, newdata = test.df)
> bagMSE3 <- mean((yhatbag3 - test.df$cnt)^2)
> bagMSE3
[1] 200567.9
```

The runtime this time is significantly better than the previous bagging attempt, however the testMSE (i.e., bagMSE 3) is bigger compared to the previous bagging attempt (200567.0 > 82640.95). We conclude that reducing the number of predictor variables will also cause the tree model to lose some of its valuable findings, since the previous bagging attempt resulted in underfitting of the training set.

Similarly, when we created a random forest tree of “cnt” on “is_holiday”, “is_weekend”, and “hour”, the runtime improved dramatically. However, the testMSE (i.e., rfMSE3) is larger compared to the previous random forest attempt (200890.4 > 124954.5).

```
rftree3 <- randomForest(cnt ~ is_holiday + is_weekend + hour, data = data, subset = train,
mtry=5, importance = TRUE)
yhatrf3 <- predict(rftree3, newdata = test.df)
rfMSE3 <- mean((yhatrf3 - test.df$cnt)^2)
rfMSE3

> yhatrf3 <- predict(rftree3, newdata = test.df)
> rfMSE3 <- mean((yhatrf3 - test.df$cnt)^2)
> rfMSE3
[1] 200890.4
```

In conclusion, our training set is too large, so using a few variables actually decreases model prediction performance. If we were to choose the decision tree as the best prediction model, the best decision tree method is the bagging approach of “cnt” on all the predictor variables.

To further improve the bagging model, we decided to change another hyperparameter in the bagging function, which is “ntree”. The default value for ntree is 500, so we decided to try a value lower and a value higher to see their effects on testMSE. First, we set “ntree = 400”, the testMSE (bagMSE1) appears to be higher than bagMSE.

```
bag1 <- randomForest(cnt~., data= data, subset = train, mtry=11, ntree = 400, importance =  
TRUE)  
bagyhat1 <- predict(bag1, newdata = test.df)  
bagMSE1 <- mean((bagyhat1 - test.df$cnt)^2)  
bagMSE1  
> bagMSE1  
[1] 82652.47
```

It appears that setting “ntree” lower causes testMSE to be larger, since bagMSE1 > bagMSE (82652.47 > 82640.95). Therefore, we decided to set “ntree” to a higher value than default, so we tried “ntree = 700”.

```
bag2 <- randomForest(cnt~., data= data, subset = train, mtry=11, ntree = 1000, importance =  
TRUE)  
bagyhat2 <- predict(bag2, newdata = test.df)  
bagMSE2 <- mean((bagyhat2 - test.df$cnt)^2)  
bagMSE2  
> bagMSE2  
[1] 82372.69
```

Once we set “ntree” to a larger number, the testMSE actually reduced. As shown in the result above, bagMSE2 < bagMSE (82372.69 < 82640.95). Although the larger the “ntree” value, the smaller the testMSE, we decided to conclude at “ntree = 1000” because the runtime of the code is taking around fifteen minutes. Consequently, the best decision tree model is bagging with all predictor variables with “mtry = 11” and “ntree = 1000”.

Performance Analysis

Because Logistic Regression and Clustering are not able to predict an exact number for count and overall are not appropriate for our dataset, we have discounted them from our final model selection. However, we wanted to consider the pros and cons of the other models we were considering.

Linear Model Pros & Cons

Overall, a Linear Model shows relationships of predictor variables on the response variable in a very easily understood manner, so in terms of interpretability, there is much less effort that goes into deriving what results mean. Furthermore, a Linear Model is also ideal for creating predictions based on predictor variables, which was one of the main goals of this project. For a business, being able to input data and get a concrete output makes it very straightforward in terms of guiding business decisions. While a Linear Model is one of the more easily understood and comprehensive models out of the five we tested, it unfortunately was not accurate to the large amount of data that we were testing for the number of new bike shares. With low R-squared values and abnormally high Test MSE, we cannot say with confidence that the quality and accuracy of the information produced by a linear model would provide significant impact for a bike-sharing company. So while we lose the ability for easy interpretability and predictive power, the trade-off is worth it because we want to ensure that a company gets accurate and actionable information about which variables will improve the count of new bike shares. Ultimately, such inaccurate data could prompt a bike-sharing company to invest in the wrong direction, negatively impacting their performance as a company.

Polynomial Model Pros & Cons

Similar to the Linear Model, a Polynomial Model allows for very concrete inputs with a specific output, which is easy to interpret and actionable for a company. However, our Polynomial Models also did not have high Multiple R-squared values paired with abnormally high Test MSE, indicating that it was also not an accurate representation of the data. So while a Polynomial Model has higher interpretability and can more easily produce predictions, we did not want to sacrifice accuracy in terms of guiding the company's knowledge of variable relationships, which would ultimately provide them with insights on how to improve bike sharing throughout the year. Because we prioritized accuracy, we ruled out the Polynomial Model because its evaluation metrics were not promising in this aspect.

Decision Trees Pros & Cons

Similar to both the Linear Model and the Polynomial Model, Decision Trees are easy to understand and interpret. In fact, they are easier to explain than Linear Model, since the tree structures can

be visualized and graphically displayed. Some also believe that Decision Trees more closely mirrors human decision making than all the other classification and regression approaches. Moreover, Decision Trees can easily handle qualitative predictors without the need to create dummy variables, so it saves a lot of human effort in preprocessing data. Unfortunately, Decision Trees generally do not have the same level of prediction accuracy as the other regression and classification models. They can also be very non-robust, where a minor change in the data can cause a significant impact on the final estimated tree.

However, aggregating many decision trees, such as methods of bagging and random forest, can substantially improve the predictive performance of the Decision Trees. Bagging helps to reduce the variance and thus increase the prediction accuracy. Plus, bagging also produces variable importance measures to help managers identify which predictor variables are the most important in influencing their target variable. Sadly, bagging results can be difficult to interpret, as it is a black box method. Similarly, random forest also has very good predictive performance, so it usually performs much better than single trees and is the top choice for predictive modeling. Random forests also produce variable importance measures for valuable business insights for managers. On the other hand, like bagging, it is also very hard to interpret the results of a random forest model, since it is also a black box method. Therefore, although random forest and bagging both improves prediction accuracy of Decision Trees, there is a major tradeoff between interpretability and prediction accuracy.

Since our goal is to predict the count of new bike shares, test mean square error (testMSE) is the most appropriate evaluation metric for regression analysis. Furthermore, as we discussed above, since we were looking for a model with the highest accuracy, we decided on Decision Trees as the most appropriate predictive model. After evaluating all our models' performance, we concluded that bagging with all predictor variables, specifically with hyperparameters of "mtry = 11" and "ntree = 1000", results in the highest prediction accuracy. The testMSE for this model is 82372.69, which is the smallest testMSE obtained from running all other models and model improvement attempts. As previously mentioned, although the testMSE appears to be abnormally high, it is normal since our training dataset is too large (~15,137 observations) and not normalized.

Critical insights obtained from our predictive models involve data size and important predictor variables influencing the count of new bike shares. Regarding data, the large volume of data we are dealing with presents a significant challenge in terms of computational power. Thus, we need more robust data handling and analysis techniques to better predict the accurate data. By investing in improved computational resources and advanced analytical tools, we can enhance our capacity for accurate predictions and more efficient data management. Finally, we observed that two variables, 'hour' and

'is_weekend', have been particularly influential in predicting bike usage patterns. Understanding these variables better allows us to tailor our services according to time-specific and weekend-related user behaviors.

Our data analysis has shown several opportunities for strategic improvements in both operational efficiency and customer engagement. By using GPS tracking, we can track and locate where bikes are most needed and adjust our fleet distribution accordingly. This allows us to identify areas of high and low usage. And this information is crucial as it guides our decisions on where to allocate bikes more effectively, ensuring availability in high-demand locations.

Additionally, gathering user feedback through our mobile app provides us with insights into why customers use our service, whether for commuting or leisure. This understanding enables us to tailor our marketing efforts more accurately and adjust bike placements to align with user preferences. Besides, implementing targeted promotions during peak hours and on days with high demand can also boost engagement significantly. These promotions could be designed to encourage usage during off-peak times as well, helping to distribute demand throughout the day.

Through these initiatives, we aim to enhance our bike share service delivery by ensuring that the bikes are available when and where they are needed most, engaging our customers to participate and share their specific needs and preferences.